

강화 학습 I

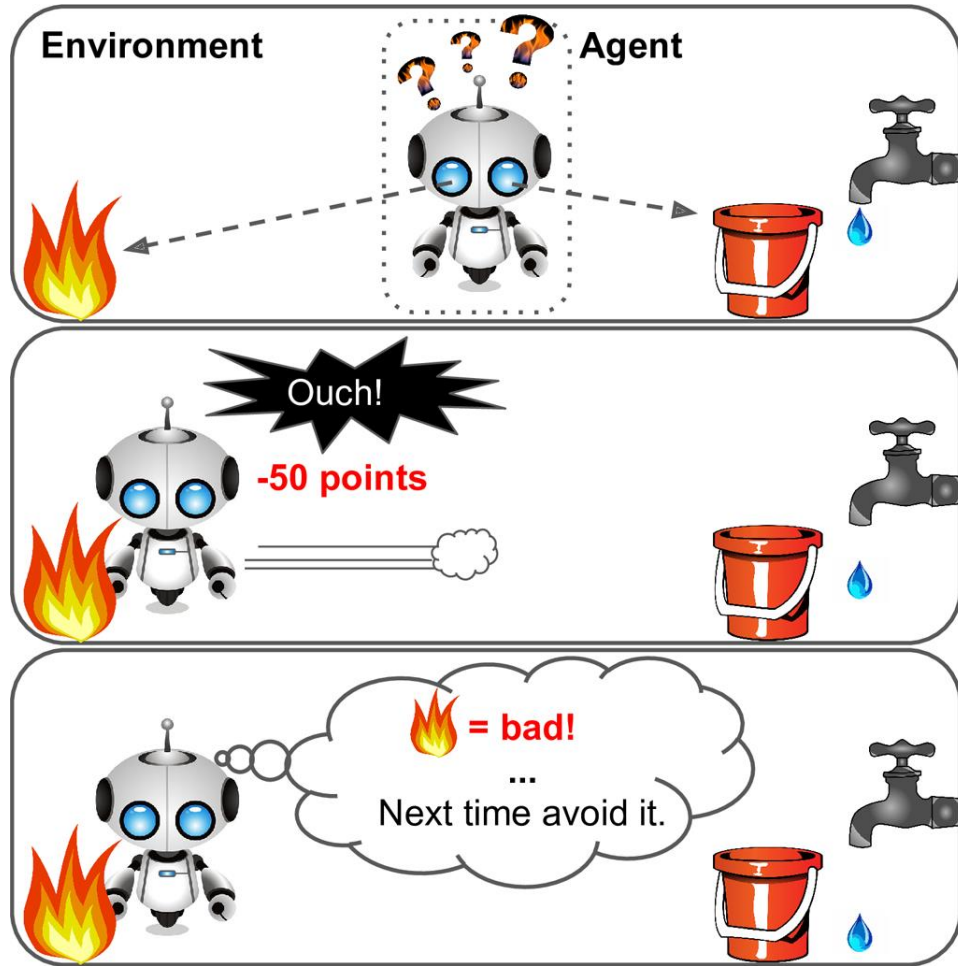
(Reinforcement learning)

강사: 김남범 교수

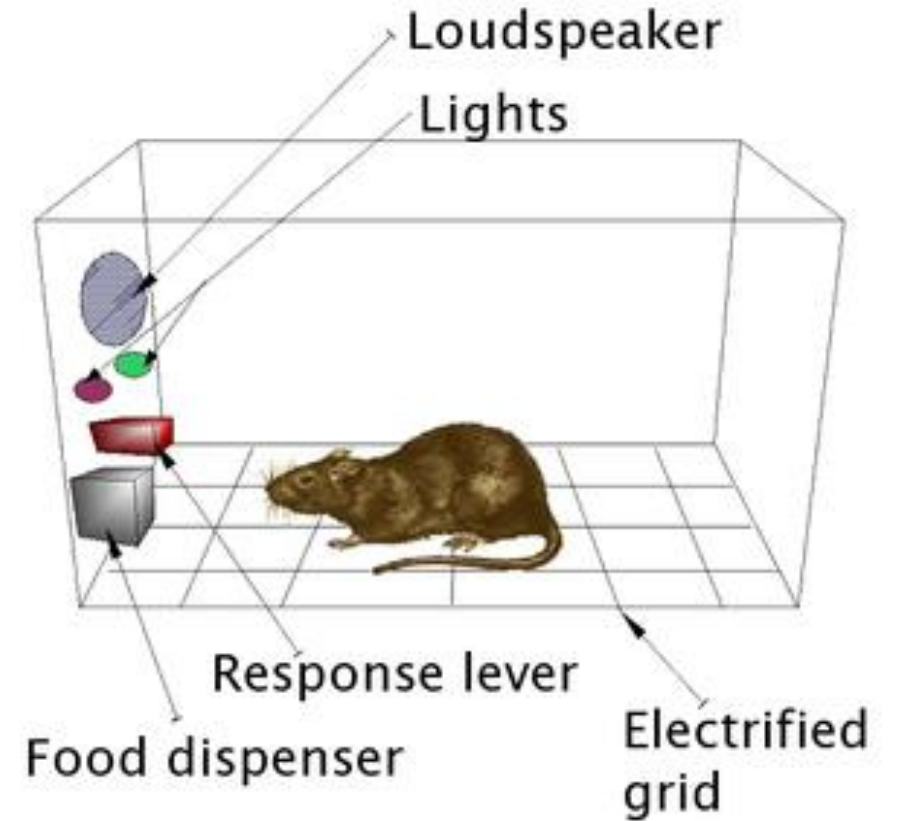
강화학습이란

- **시행착오(Trial and Error)**와 **보상(Reward)** 을 통해 학습을 하여 목표를 찾아가는 알고리즘.

강화학습 예제

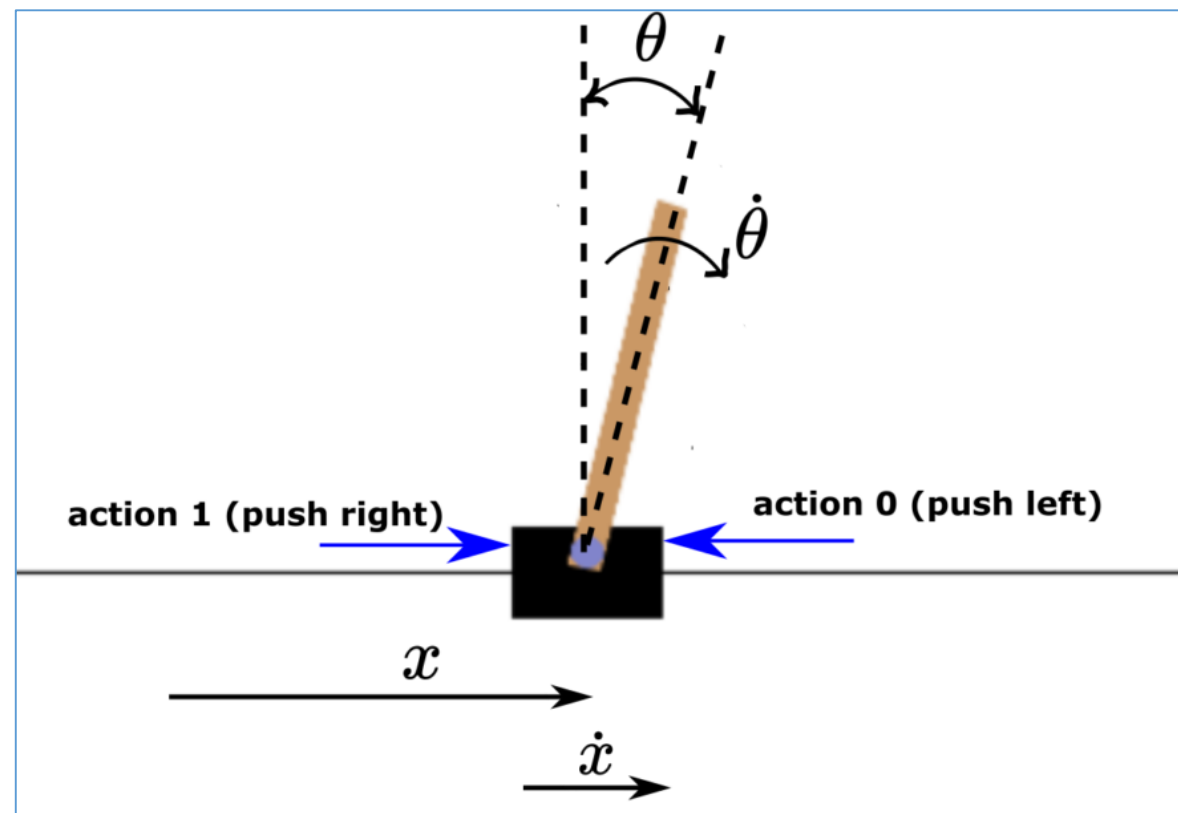


- 1 Observe
- 2 Select action using policy
- 3 Action!
- 4 Get reward or penalty
- 5 Update policy (learning step)
- 6 Iterate until an optimal policy is found





Cart Pole



FrozenLake 환경이해

Environment

Action Space	Discrete(4)
Observation Space	Discrete(16)
import	<code>gymnasium.make("FrozenLake-v1")</code>



Observation Space [¶](#)

The observation is a value representing the player's current position as $\text{current_row} * \text{ncols} + \text{current_col}$ (where both the row and col start at 0).

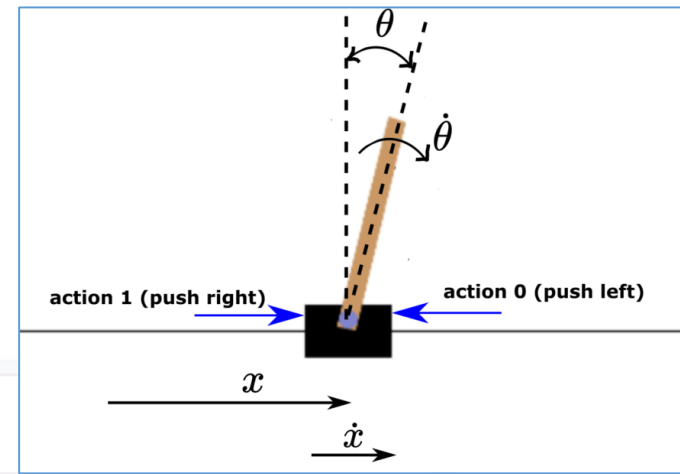
For example, the goal position in the 4x4 map can be calculated as follows: $3 * 4 + 3 = 15$. The number of possible observations is dependent on the size of the map.

The observation is returned as an `int()`.

Cart Pole 환경이해

Environment

Action Space	<code>Discrete(2)</code>
Observation Space	<code>Box([-4.8 -inf -0.41887903 -inf], [4.8 inf 0.41887903 inf], (4,), float32)</code>
import	<code>gymnasium.make("CartPole-v1")</code>

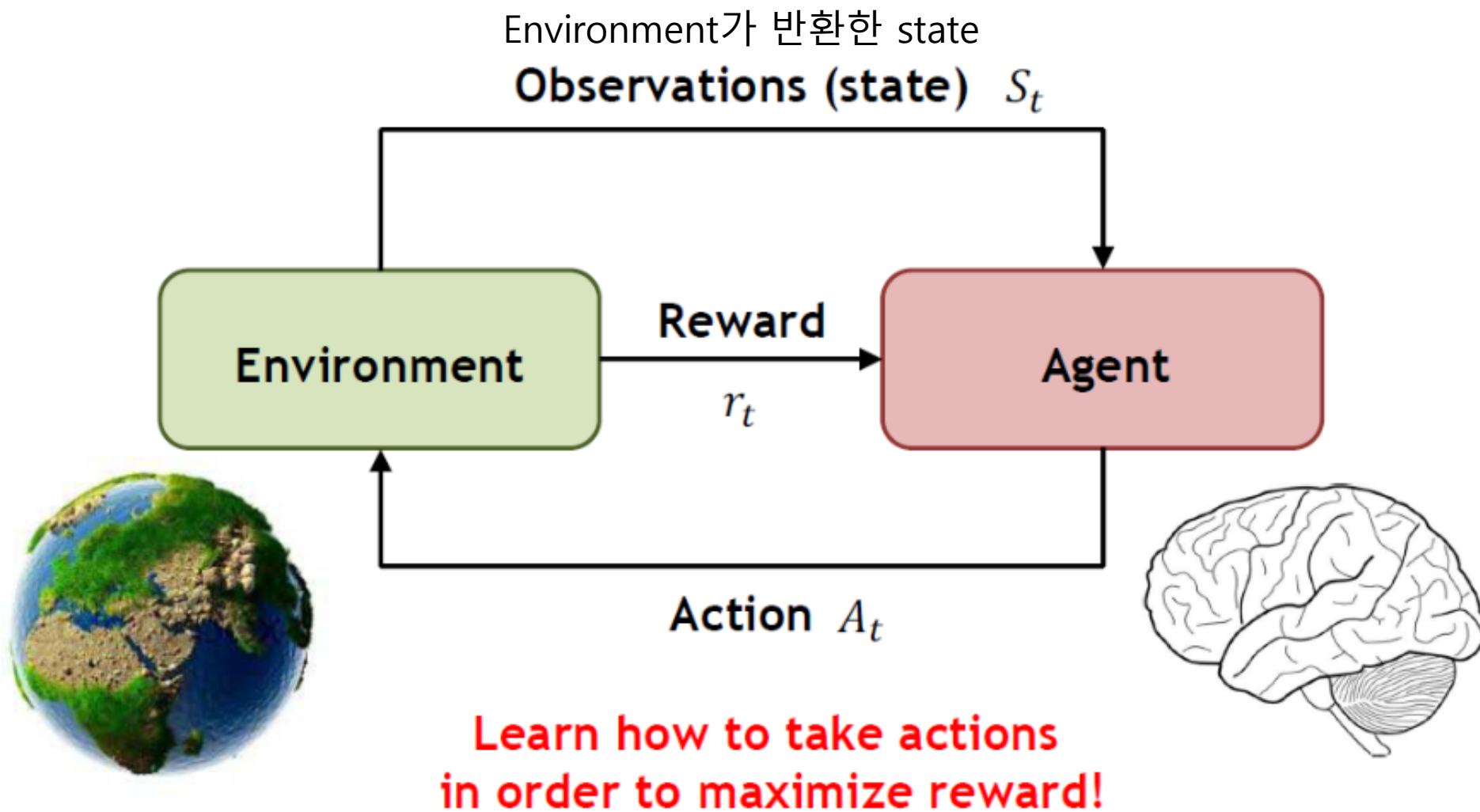


Observation Space ¶

The observation is a `ndarray` with shape `(4,)` with the values corresponding to the following positions and velocities:

Num	Observation	Min	Max
0	Cart Position	-4.8	4.8
1	Cart Velocity	-Inf	Inf
2	Pole Angle	~ -0.418 rad (-24°)	~ 0.418 rad (24°)
3	Pole Angular Velocity	-Inf	Inf

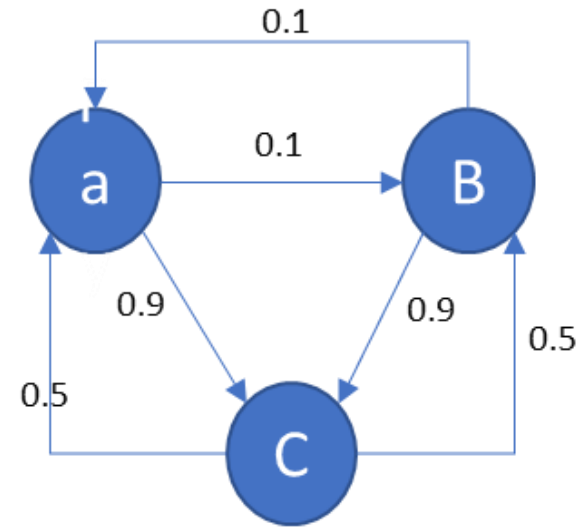
에이전트와 환경



환경과 모델 (Model)

- Model-based

- 각 state에서 상태전이 확률을 알고 있음
- 각 state에서 보상을 알고 있음

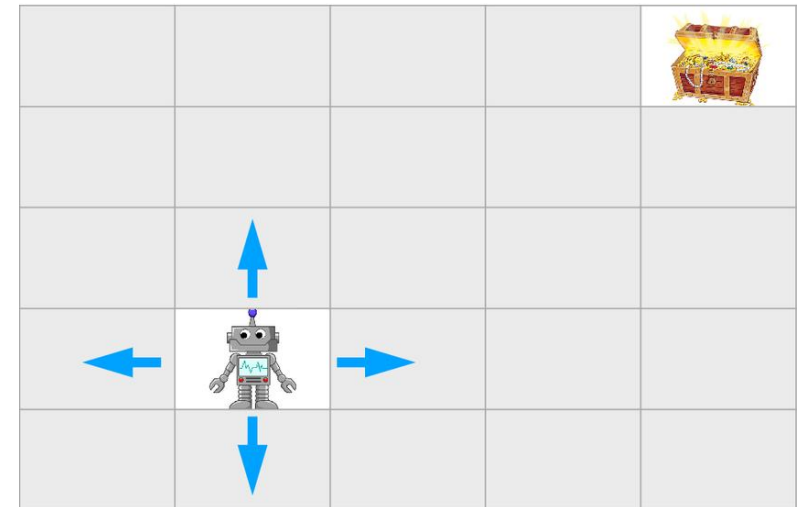


J

0	0.1	0.9
0.9	0.0	0.1
0.5	0.5	0.0

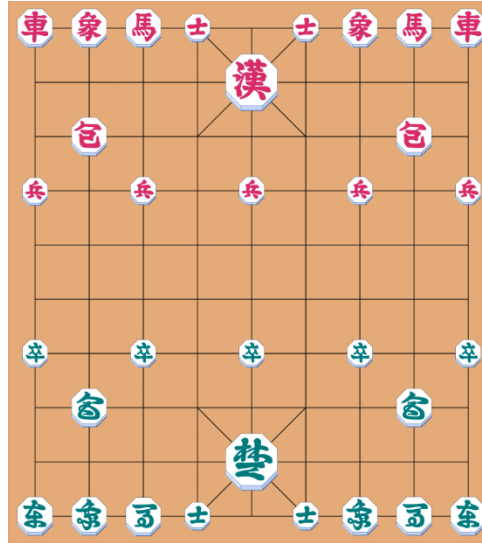
- Model-free

- 각 state에서 상태전이 확률을 모르고 있음
- 각 state에서 보상을 모르고 있음



Reward 표현방식

- Goal reward
 - goal = 1
- Action penalty
 - step = -1
 - goal = 0



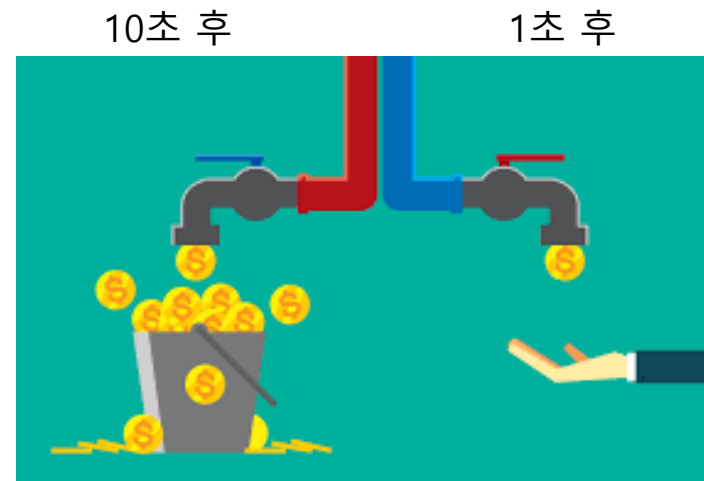
	1	2	3	4	5
1					
2	●				
3			+5		
4					
5					+10

state and observation

- State는 environment의 현재 상태에 대한 모든 정보
- Observation은 state의 부분집합
- state 와 observation은 같은 개념으로 사용
- 기호: s_t

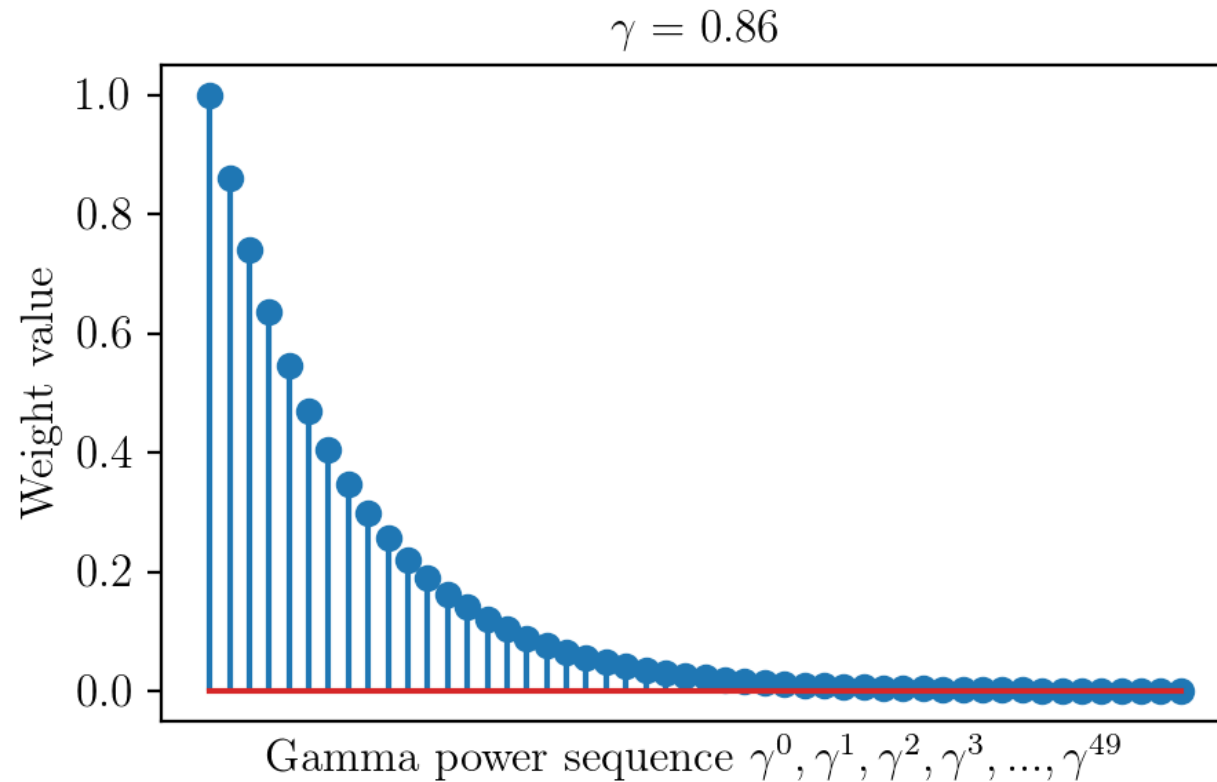
보상 (Reward) vs 가치함수 (Value function)

- 보상 (Reward)
 - 즉각적인 보상
 - r_t : t시점에서 얻게 되는 보상
- 이득 (Return ~ Gain)
 - Time t 이후의 reward의 총합
 - $G_t = r_{t+1} + \gamma * r_{t+2} + ... + \gamma^{T-1} * r_T$
- 가치함수 (Value function)
 - 장기적인 관점의 누적보상의 기댓값
 - $E[G_t] = E[r_{t+1} + \gamma * r_{t+2} + ... + \gamma^{T-1} * r_T]$



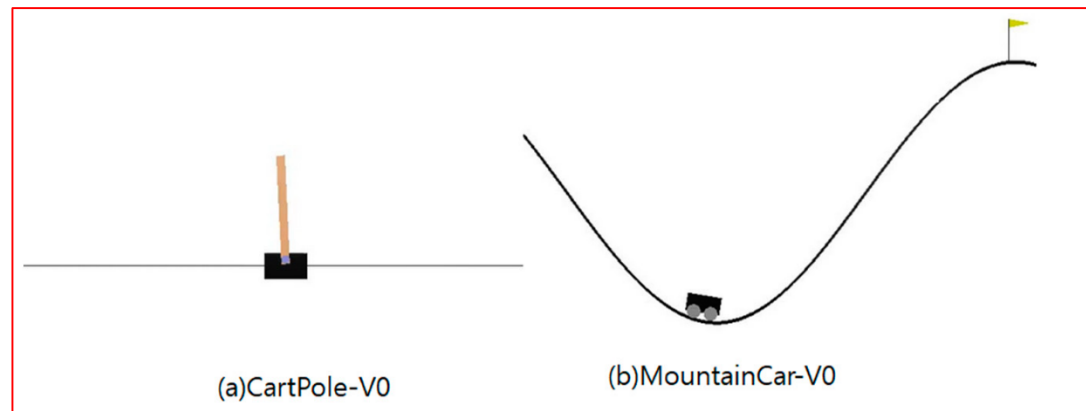
Discounting factor

- $G_t = r_{t+1} + \gamma * r_{t+2} + \dots + \gamma^{T-1} * r_T$



행동 (Action)

- Discrete action
 - 예) Frozen lake
- Continuous action
 - 예) CartPole
 - MountainCar
- 기호: a_t



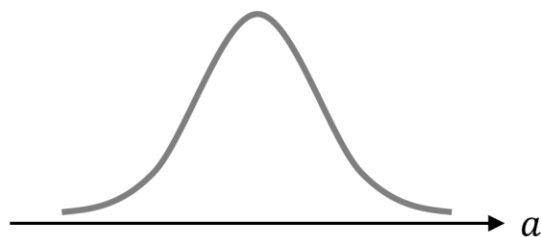
정책 (Policy)

- 결정론적 (Deterministic)

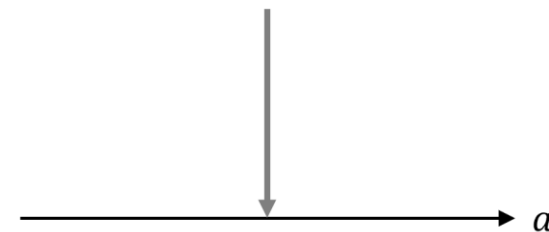
- $\pi(a_t|s_t) = k$

- 확률론적 (Stochastic)

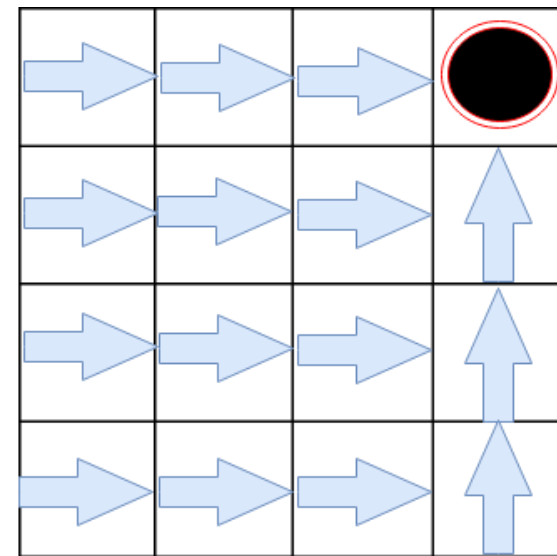
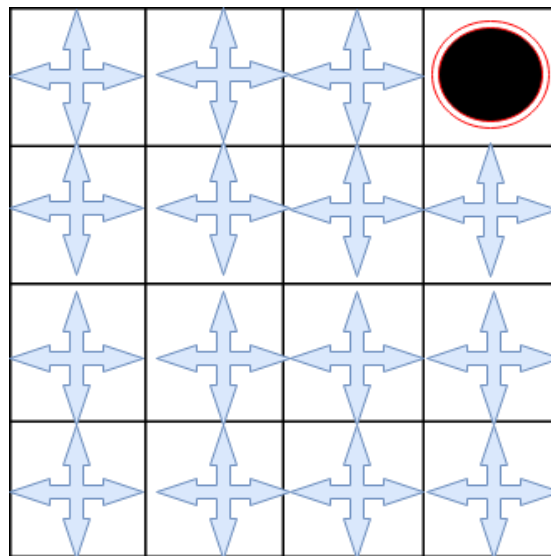
- $\pi(a_t|s_t) \sim P(a_t|s_t, \theta)$



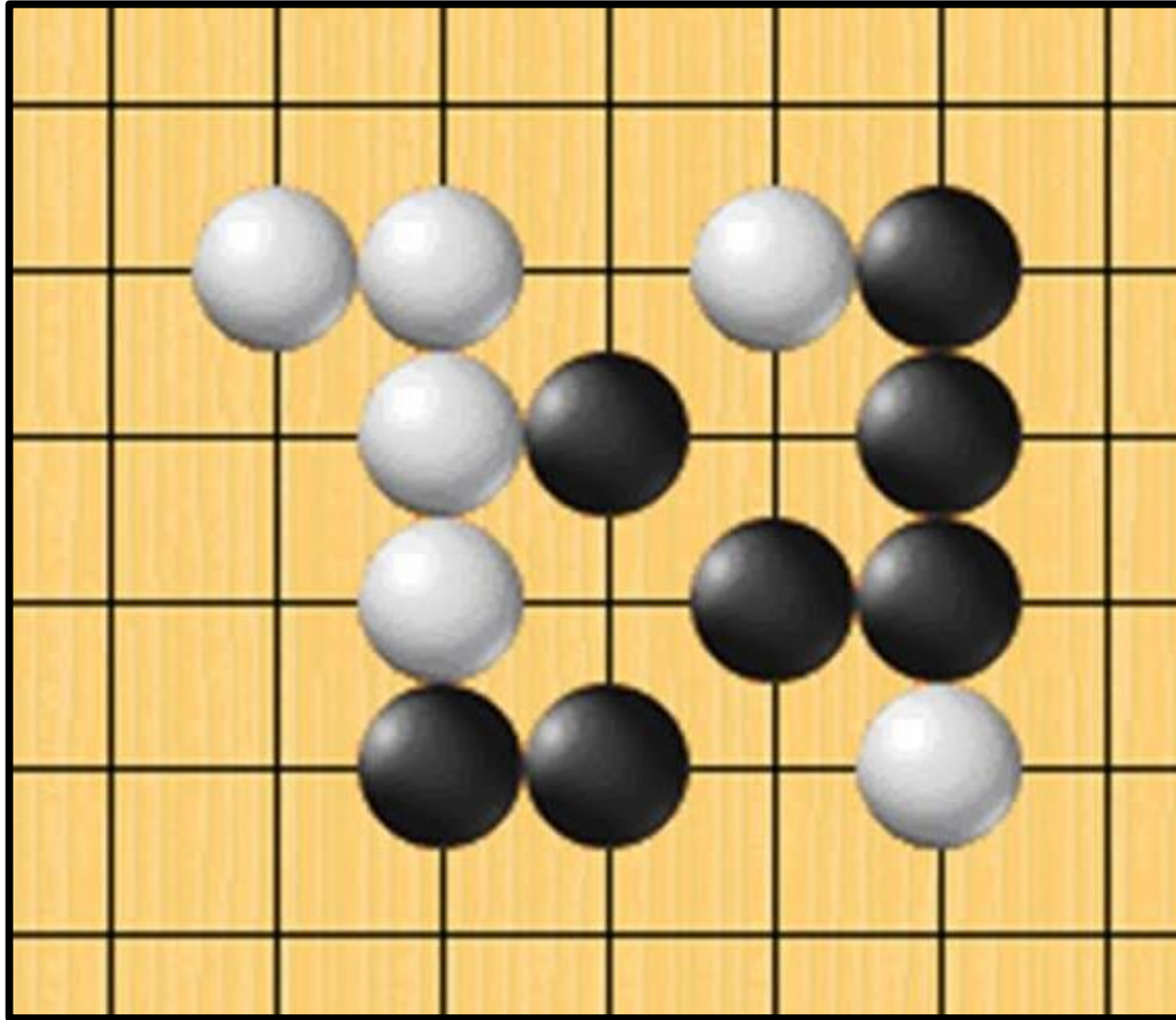
Stochastic policy
 $\pi_{\theta}(a|s) = P[a|s; \theta]$



Deterministic policy
 $a = \mu_{\theta}(s)$

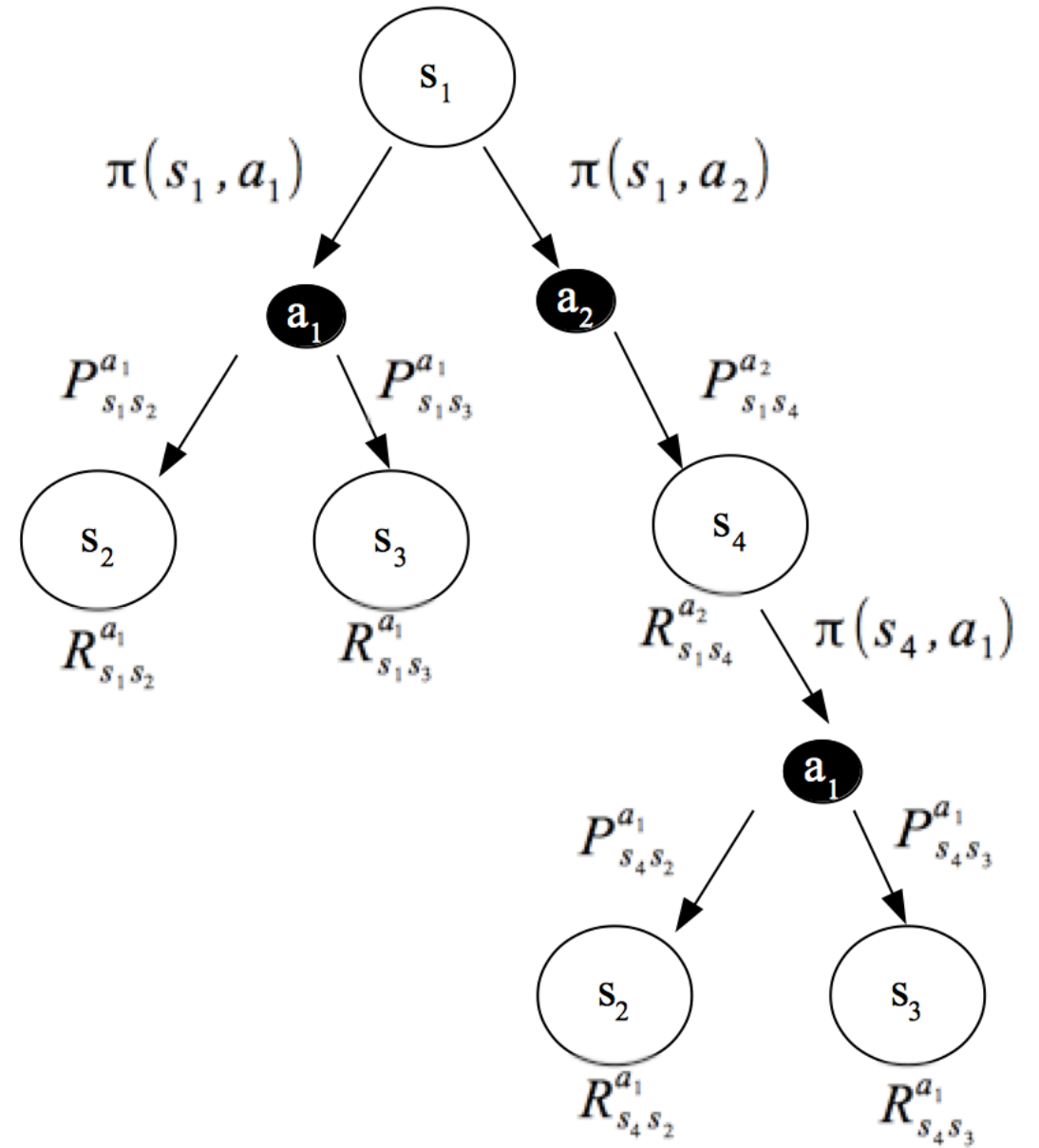


결정론적 대 확률론적 환경



Stochastic environment

Slippery = True



Task의 종류

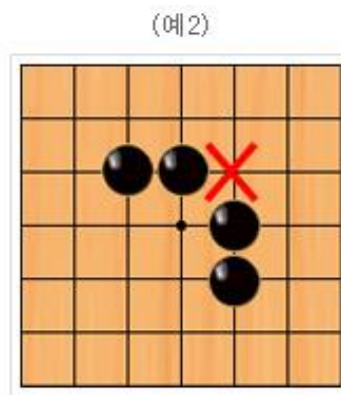
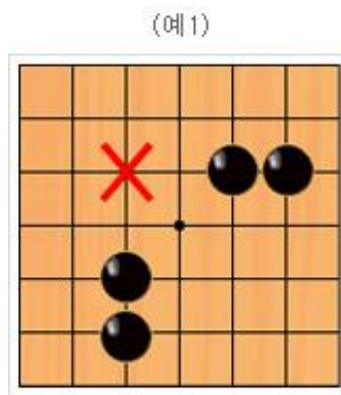
- Episodic task

- 오목

- Agent – player
 - Environment – 바둑판 전체 (program)
 - State (observation)
 - Action – 어떤 위치에 바둑돌을 놓는 것
 - Reward – 승리(1), 실패(-1)

- Continuing task

- 전력량 최적화
 - 빌딩 에어컨



Dynamic programming (1953)

- Markov decision process (MDP)
- Bellman equation

Markov state vs Non-Markov state

Markov state



Non - Markov state

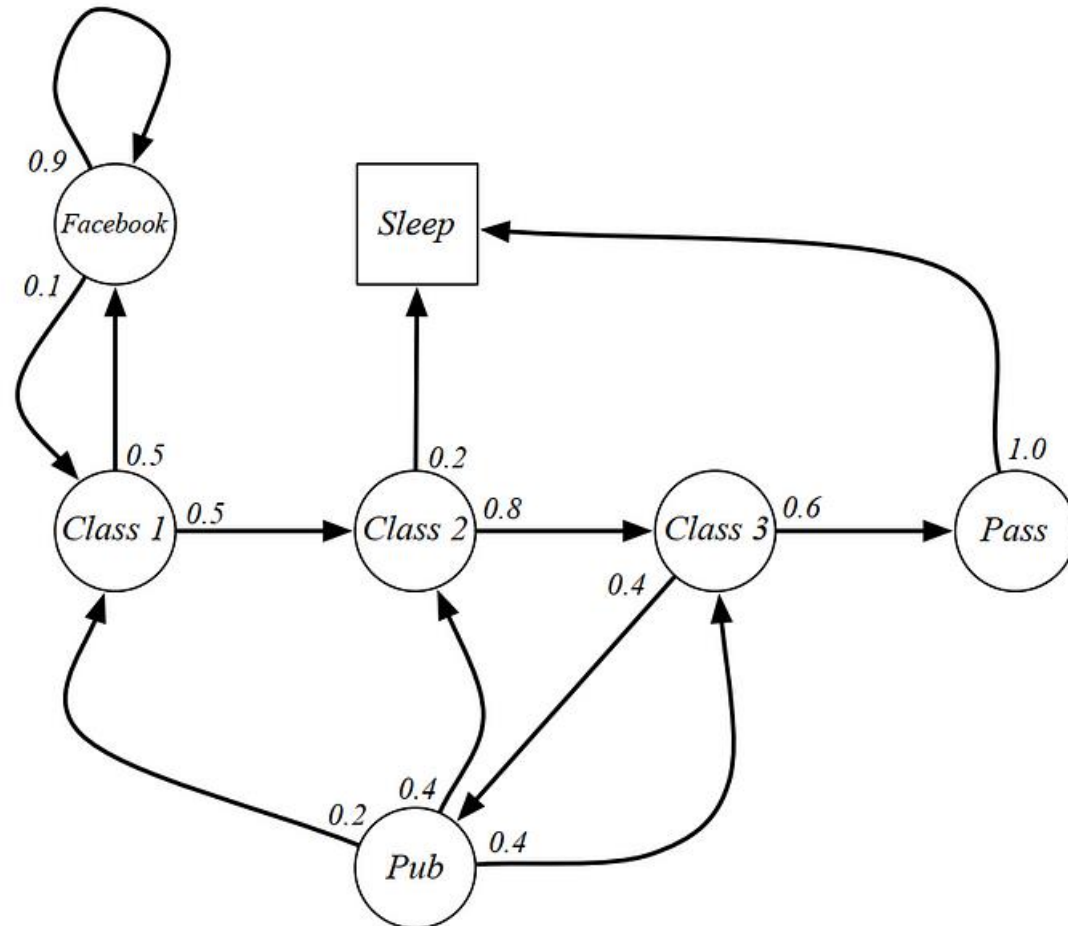


Markov Process (MP) or Markov Chain

MP (S, P)

상태전이확률 :

State transition probability



Markov property

$$Pr(S_{t+1}=s' \mid S_0, S_1, \dots, S_{t-1}, S_t) = Pr(S_{t+1}=s' \mid S_t)$$

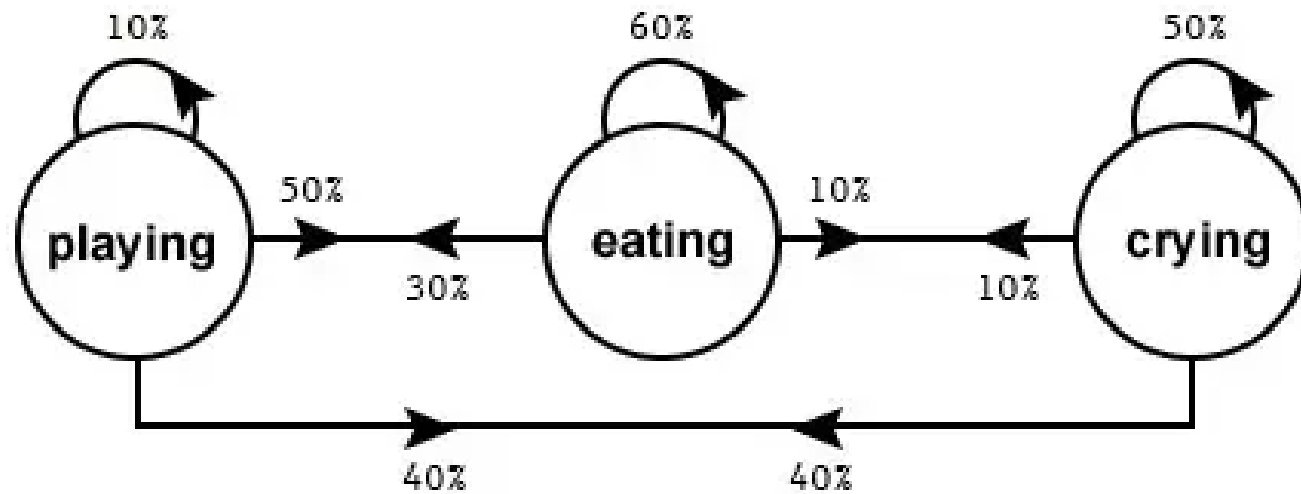
State transition matrix

$\mathcal{P} =$

	Class 1	Class 2	Class 3	SNS	Pass	Pub	Sleep
Class 1		0.5		0.5			
Class 2			0.8				0.2
Class 3					0.6	0.4	
SNS	0.1			0.9			
Pass							1
Pub	0.2	0.4	0.4				
Sleep							1

Markov Process (MP) 예제

Markov state diagram of a child behaviour



State transition matrix

Markov reward process (MRP)

MP (S , P , R , γ)

- Reward r_t
- Discounting factor $\gamma \in [0,1]$
- Return: total discounted reward $G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} \dots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$
- **State-value function**

$$V(s) = \mathbb{E}[G_t | S_t = s]$$

$\langle S, P_{tran}, R, \gamma \rangle$

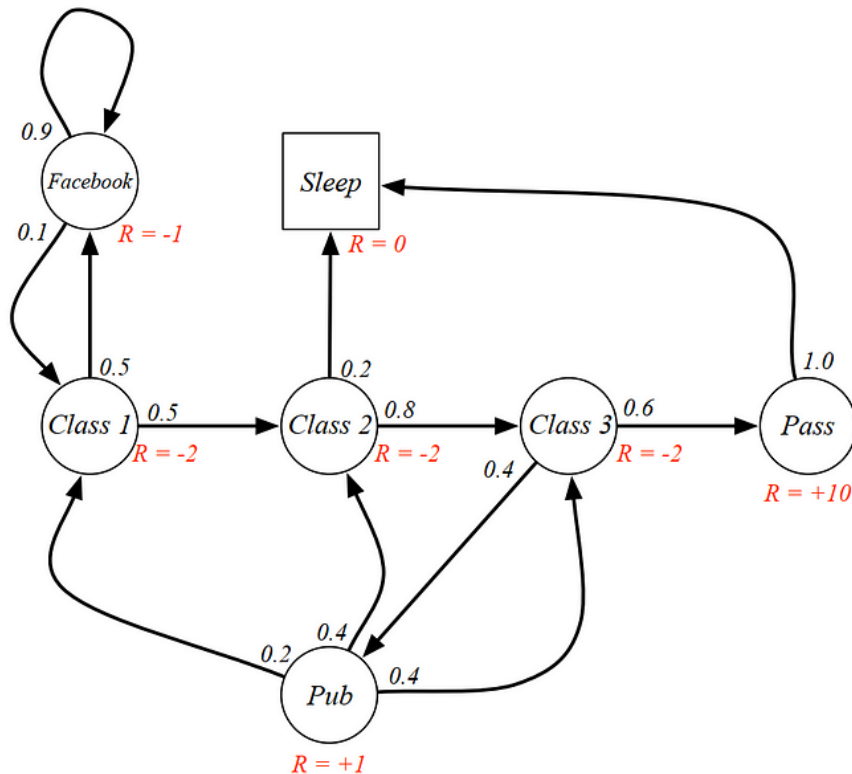
S : set of states

P_{tran} : state transition probability matrix

R : Reward function

γ : Discounting factor

$\gamma = 0$: Short sighted agent
 $\gamma = 1$: Far sighted agent
 $0 < \gamma < 1$: 일반적



Markov decision process (MDP)

MP (S , A , P , R , γ)

- Action A_t



- Policy $\pi(a|s) = P(A_t = a|S_t = s)$
- Action-value function $Q_\pi(s, a) = \mathbb{E}_\pi[G_t|S_t = s, A_t = a]$

$\langle S, A, P_{tran}, R, \gamma \rangle$

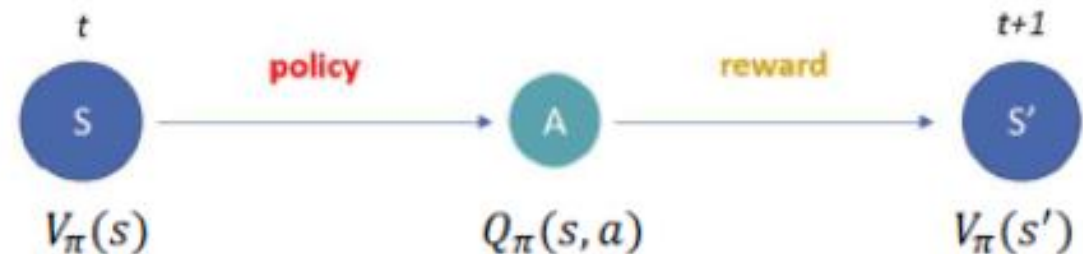
S : set of states

A : set of actions

P_{tran} : state transition probability matrix

R : Reward function

γ : Discounting factor



가치 함수 (Value function)

상태가치 함수

Definition

The **state-value function** $v_{\pi}(s)$ of an MDP is the expected return starting from state s , and then following policy π

$$v_{\pi}(s) = \mathbb{E}_{\pi} [G_t \mid S_t = s]$$

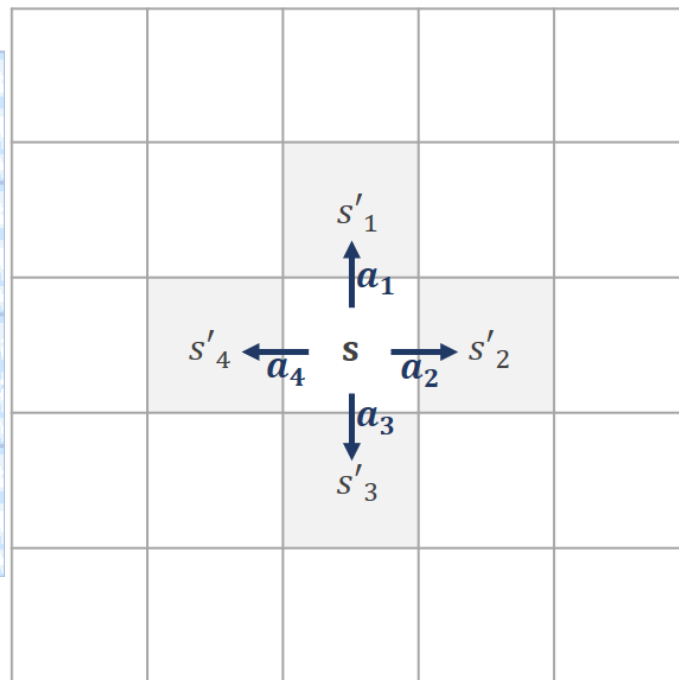
행동가치 함수

Definition

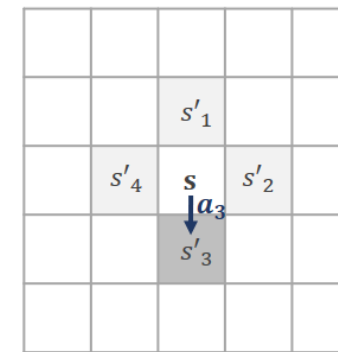
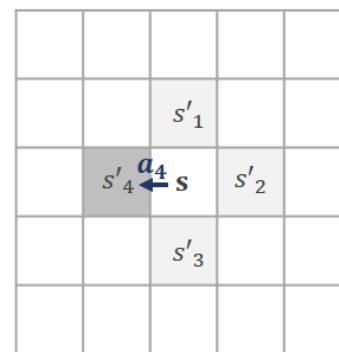
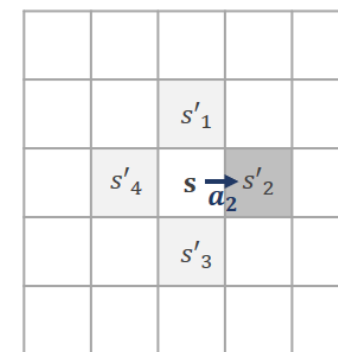
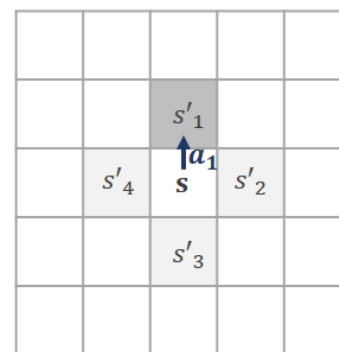
The **action-value function** $q_{\pi}(s, a)$ is the expected return starting from state s , taking action a , and then following policy π

$$q_{\pi}(s, a) = \mathbb{E}_{\pi} [G_t \mid S_t = s, A_t = a]$$

상태 가치함수 vs 행동 가치함수



상태 가치함수



각 행동에 대한 행동 가치함수

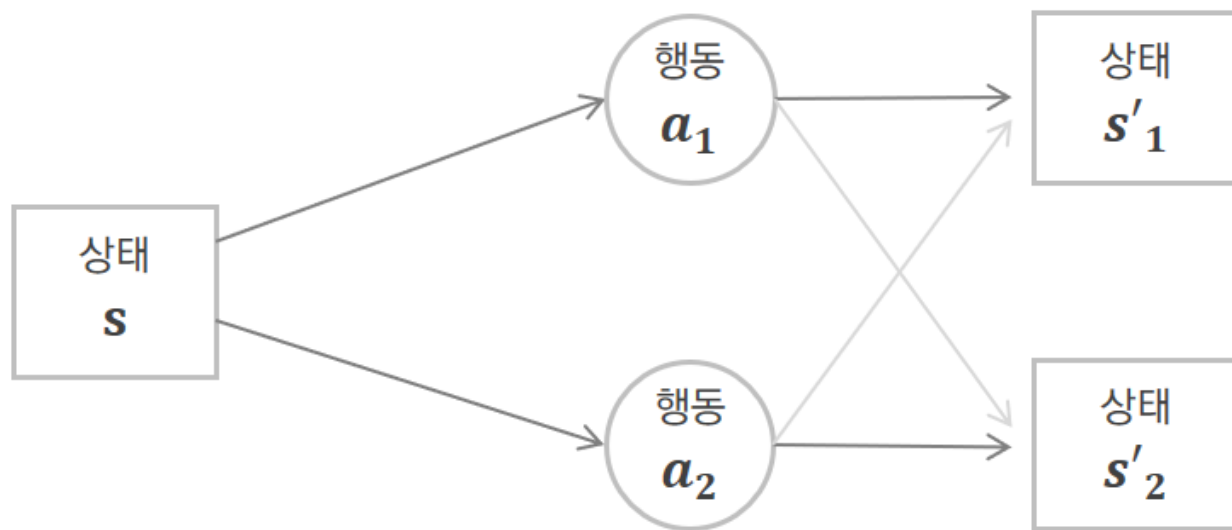
모든 행동에 대해 각 행동이 일어날 확률 곱해서 더함

$$v_{\pi}(s) = E_{\pi}[q_{\pi}(s, a) | S_t = s] = \sum_{a \in A} \pi(a|s) q_{\pi}(s, a)$$

행동가치 함수 (q 함수)

$$q_{\pi}(s, a) = E_{\pi}[G_t | S_t = s, A_t = a]$$

$$q_{\pi}(s, a_1) = E_{\pi}[G_t | S_t = s, A_t = a_1]$$

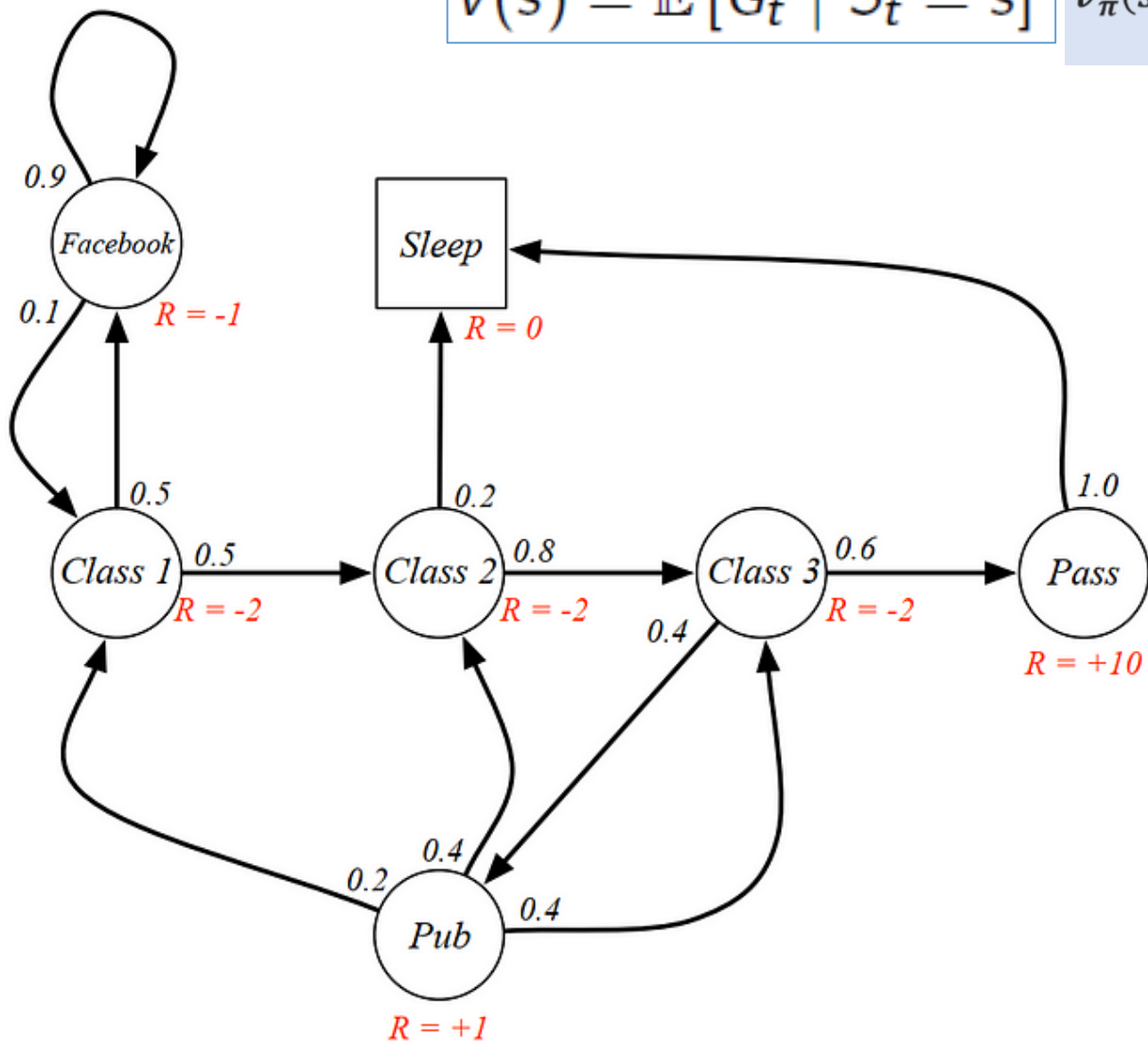


$$q_{\pi}(s, a_2) = E_{\pi}[G_t | S_t = s, A_t = a_2]$$

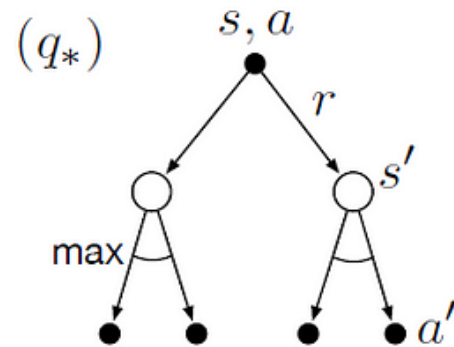
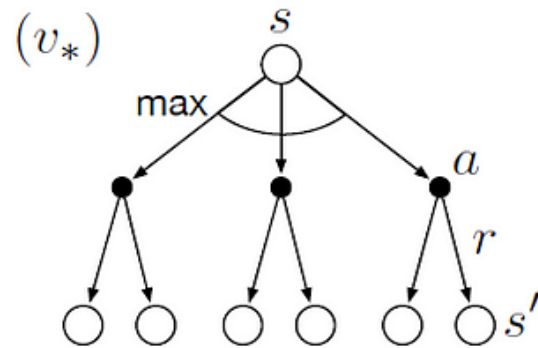
상태가치함수 예제

$$v(s) = \mathbb{E}[G_t \mid S_t = s]$$

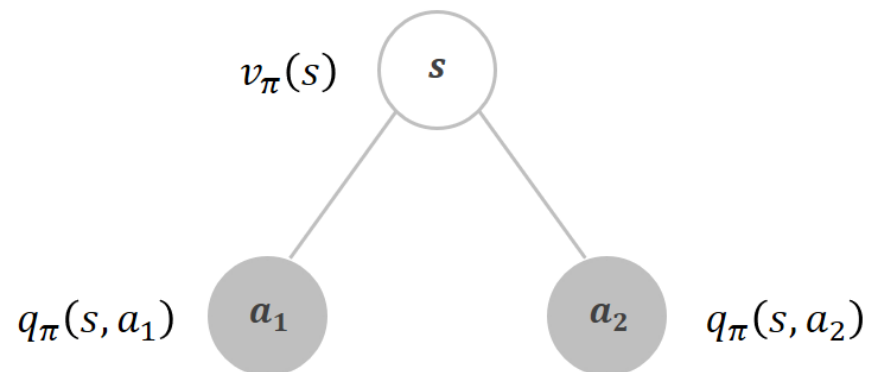
$$v_{\pi}(s) = E_{\pi}[q_{\pi}(s, a) \mid S_t = s] = \sum_{a \in A} \pi(a|s) q_{\pi}(s, a)$$



Backup diagram



Backup diagrams for v_* and q_*



$$v_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a|s) q_\pi(s, a)$$

Bellman Expectation Equation :

상태 가치함수 ($V(s)$, State-value function)

The **value function** can be decomposed into two parts:

- immediate reward R_{t+1}
- discounted value of successor state $\gamma v(S_{t+1})$

$$\begin{aligned} v(s) &= \mathbb{E}[G_t \mid S_t = s] \\ &= \mathbb{E}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \mid S_t = s] \\ &= \mathbb{E}[R_{t+1} + \gamma (R_{t+2} + \gamma R_{t+3} + \dots) \mid S_t = s] \\ &= \mathbb{E}[R_{t+1} + \gamma G_{t+1} \mid S_t = s] \\ &= \mathbb{E}[R_{t+1} + \gamma v(S_{t+1}) \mid S_t = s] \end{aligned}$$

상태가치 함수 (State value function) 와 policy

$$v_{\pi}(s) \doteq \mathbb{E}_{\pi}[G_t \mid S_t = s] \text{ MC method}$$

$$= \mathbb{E}_{\pi}[R_{t+1} + \gamma G_{t+1} \mid S_t = s] \text{ TD method}$$

$$= \sum_a \pi(a|s) \sum_{s'} \sum_r p(s', r|s, a) \left[r + \gamma \mathbb{E}_{\pi}[G_{t+1} | S_{t+1} = s'] \right]$$

$$= \sum_a \pi(a|s) \sum_{s', r} p(s', r|s, a) \left[r + \gamma v_{\pi}(s') \right], \quad \text{for all } s \in \mathcal{S},$$

벨만 최적 방정식

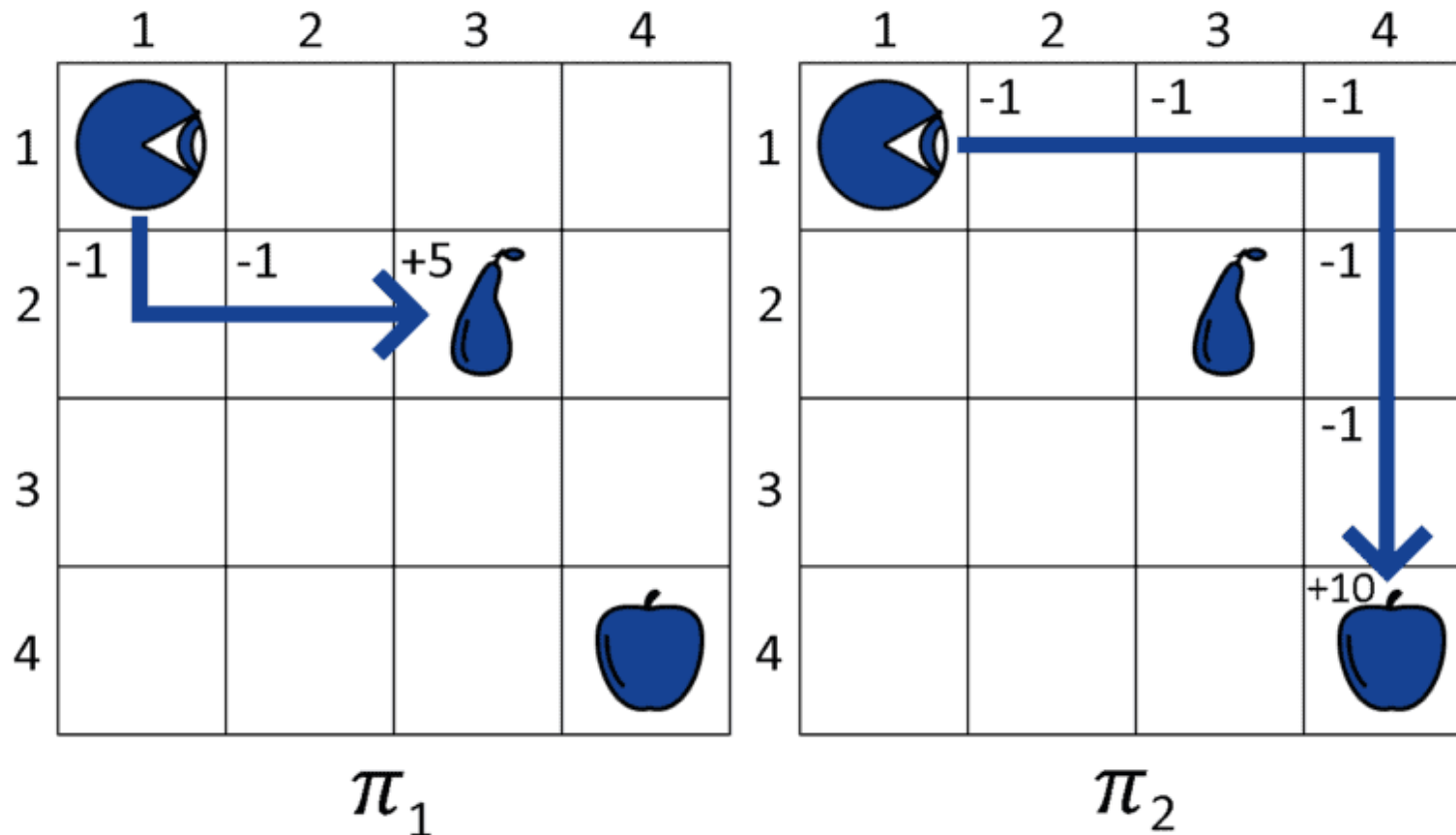
$$v_*(s) = \max_{\pi} v_{\pi}(s)$$

$$q_*(s, a) = \max_{\pi} q_{\pi}(s, a)$$

$$Q_{\pi}(s, a) = q_{\pi}(s, a) = \sum_{s', r} p(s', r|s, a) [r + \gamma V_{\pi}(s')]$$

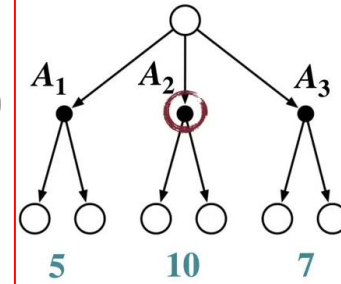
좋은 정책 (Better Policy)

$$\pi(a|s) = \mathbb{P}[A_t = a | S_t = s] \quad \pi_2 \text{ is better than } \pi_1$$



최적 정책 (Optimal Policy)

Determining an Optimal Policy



$$v_*(s) = \max_a \sum_{s'} \sum_r p(s', r | s, a) [r + \gamma v_*(s')]$$
$$\pi_*(s) = \operatorname{argmax}_a \sum_{s'} \sum_r p(s', r | s, a) [r + \gamma v_*(s')]$$



Dynamic programming

- Policy evaluation (정책평가)
 - 정책평가 – state value update
- Policy iteration (정책반복)
 - 정책평가 (evaluation)
 - 정책향상 (improvement) e.g., Optimal policy
- Value iteration (가치 반복)
 - 정책평가 + 정책향상을 동시

$$\begin{aligned} v_{\pi}(s) &\doteq \mathbb{E}_{\pi}[G_t \mid S_t = s] \\ &= \mathbb{E}_{\pi}[R_{t+1} + \gamma G_{t+1} \mid S_t = s] \\ &= \sum_a \pi(a|s) \sum_{s'} \sum_r p(s', r|s, a) \left[r + \gamma \mathbb{E}_{\pi}[G_{t+1} | S_{t+1} = s'] \right] \\ &= \sum_a \pi(a|s) \sum_{s', r} p(s', r|s, a) \left[r + \gamma v_{\pi}(s') \right], \quad \text{for all } s \in \mathcal{S}, \end{aligned}$$

정책평가 (Iterative policy evaluation)

$$\sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) \left[r + \gamma v_{\pi}(s') \right], \quad \text{for all } s \in \mathcal{S},$$

V

출발 s0 T	s1	s2	s3
s4	s5	s6	s7
s8	s9	s10	s11
s12	s13	s14	도착 s15 T

V'

출발 s0 T	s1	s2	s3
s4	s5	s6	s7
s8	s9	s10	s11
s12	s13	s14	도착 s15 T

Iterative Policy Evaluation, for estimating $V \approx v_\pi$

Input π , the policy to be evaluated

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation

Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(\text{terminal}) = 0$

Loop:

$\Delta \leftarrow 0$

 Loop for each $s \in \mathcal{S}$:

$v \leftarrow V(s)$

$V(s) \leftarrow \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

until $\Delta < \theta$

정책반복 (Iterative policy iteration)

V_k for the Random Policy

Greedy Policy w.r.t. V_k

random policy

greedy policy

$k = 0$

0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0

$k = 1$

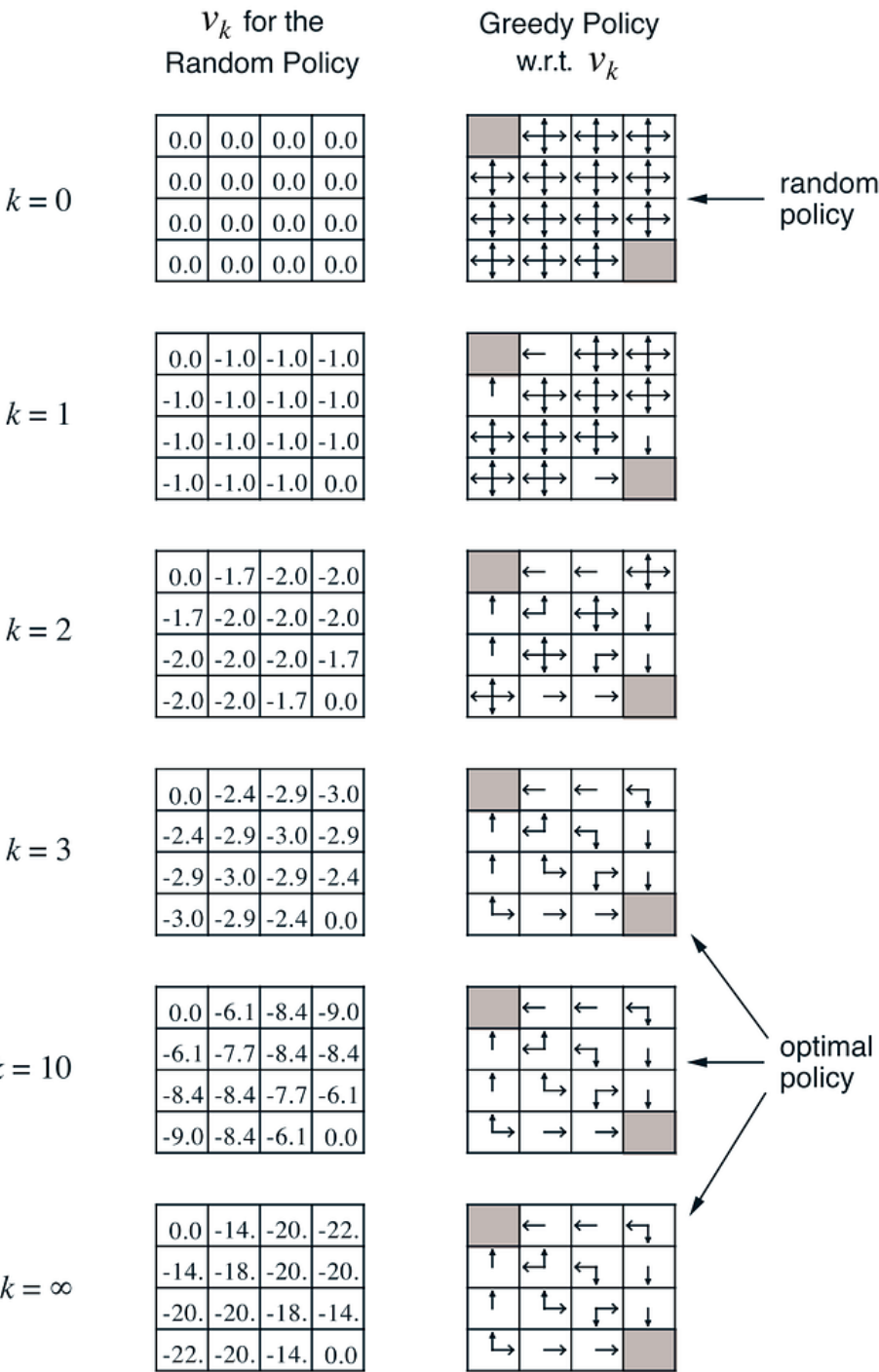
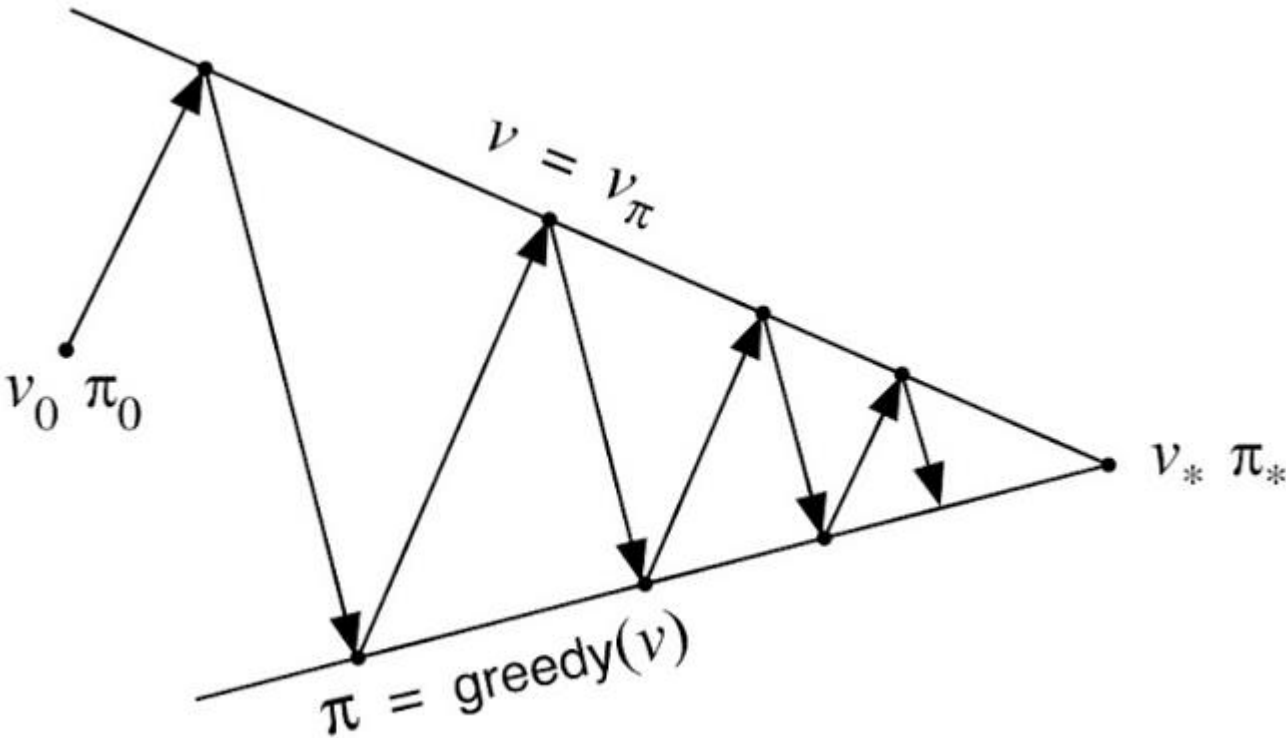
0.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	0.0

$k = 2$

0.0	-1.7	-2.0	-2.0
-1.7	-2.0	-2.0	-2.0
-2.0	-2.0	-2.0	-1.7
-2.0	-2.0	-1.7	0.0

정책반복의 수렴 (Iterative policy iteration convergence)

the environment is a finite Markov Decision Process (finite MDP)



정책반복 알고리즘 (Policy iteration pseudocode)

```
input : reward function  $r(s)$ , transitional model  $p(s'|s, a)$ ,  
        discounted factor  $\gamma$ , convergence threshold  $\theta$   
        policy  $\pi(s)$ , value  $v(s)$   
output: converged value  $v(s)$   
1 converge  $\leftarrow$  false  
2 while converge = false do  
3    $\Delta \leftarrow 0$   
4   for  $s \in S$  do  
5     temp  $\leftarrow v(s)$   
6      $v(s) \leftarrow r(s) + \gamma \sum_{s'} p(s'|s, a = \pi(s))v(s')$   
7      $\Delta \leftarrow \max(\Delta, |\text{temp} - v(s)|)$   
8   end  
9   if  $\Delta < \theta$  then  
10    converge  $\leftarrow$  true  
11  end  
12 end  
13 return  $v(s)$ 
```

```
input : transitional model  $p(s'|s, a)$ ,  
        policy  $\pi(s)$ , value  $v(s)$   
output: updated policy  $\pi(s)$ ,  
        binary indicating whether any change occurs  
1 change  $\leftarrow$  false  
2 for  $s \in S$  do  
3   temp  $\leftarrow \pi(s)$   
4    $\pi(s) \leftarrow \operatorname{argmax}_a \sum_{s'} p(s'|s, a)v(s')$   
5   if temp  $\neq \pi(s)$  then  
6     change  $\leftarrow$  true  
7   end  
8 end  
9 return  $\pi(s)$ , change
```

가치반복 (Iterative value iteration)

Initialize $V(s)$ arbitrarily, for all $s \in \mathcal{S}$

Initialize θ to a small positive value

Loop:

$\Delta \leftarrow 0$

Loop for each $s \in \mathcal{S}$:

$v \leftarrow V(s)$

$V(s) \leftarrow \max_a \sum_{s',r} p(s', r | s, a) [r + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

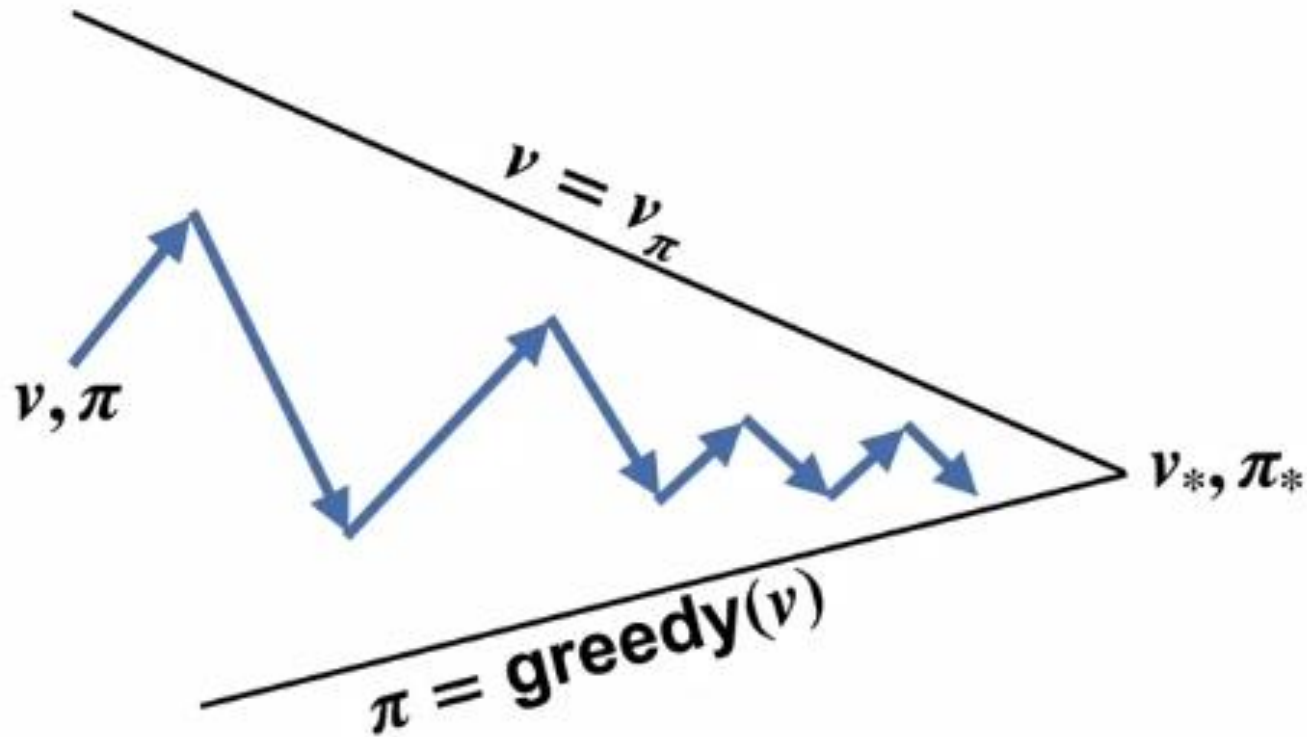
Until $\Delta < \theta$

Output a deterministic policy, $\pi \approx \pi^*$, such that

$\pi(s) \leftarrow \operatorname{argmax}_a \sum_{s',r} p(s', r | s, a) [r + \gamma V(s')]$

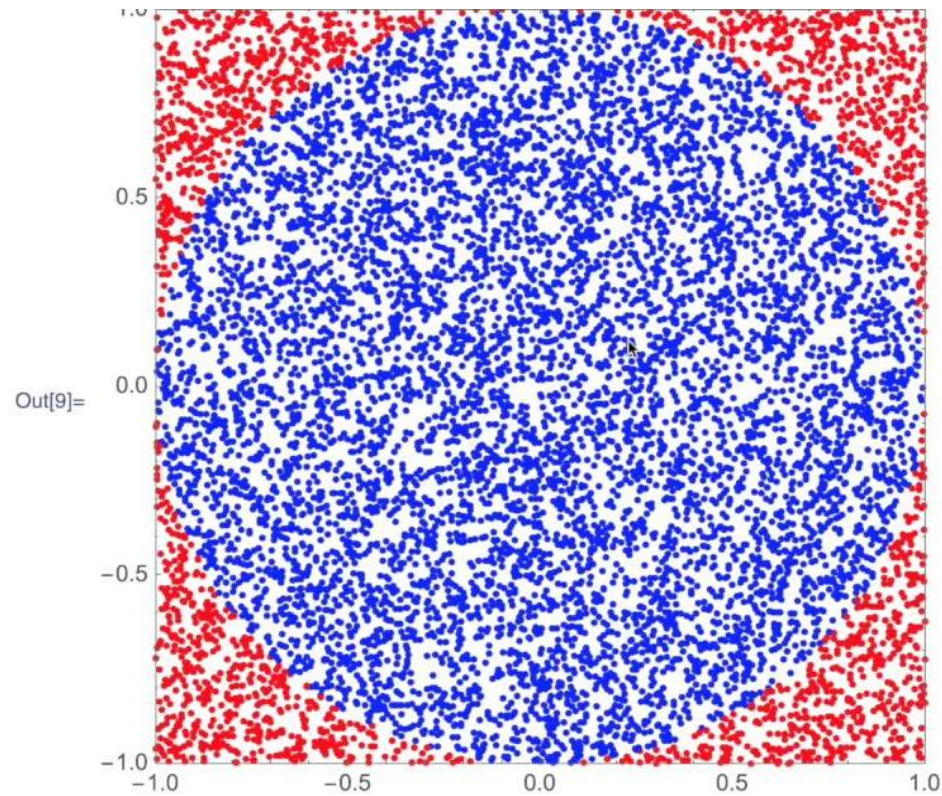
Generalized policy iteration

Iterative value iteration

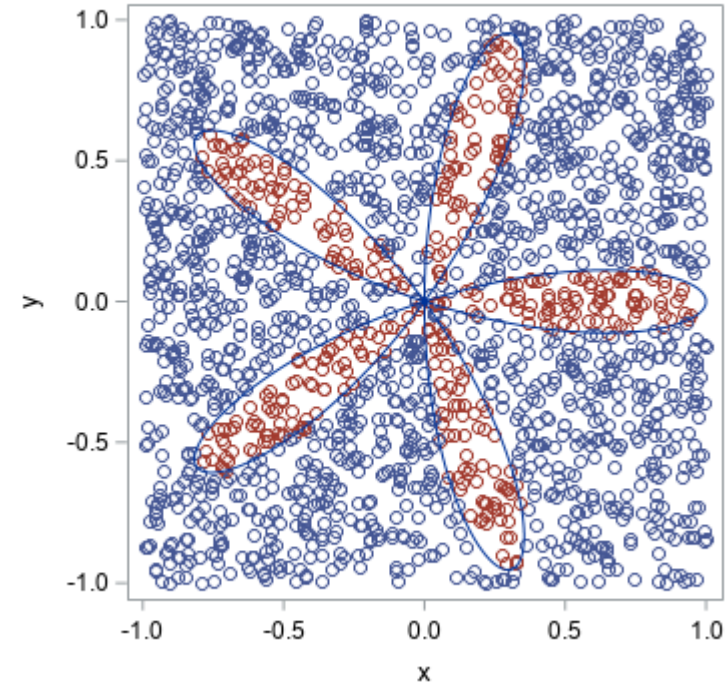


Monte-Carlo method란

Circle area



Monte Carlo Estimate of Area
Polar Rose: $r = \cos(5 \theta)$



Monte-Carlo method for policy evaluation

$$\begin{aligned} v_{\pi}(s) &\doteq \mathbb{E}_{\pi}[G_t \mid S_t = s] \text{ MC에서 이용} \\ &= \mathbb{E}_{\pi}[R_{t+1} + \gamma G_{t+1} \mid S_t = s] \\ &= \sum_a \pi(a|s) \sum_{s'} \sum_r p(s', r | s, a) \left[r + \gamma \mathbb{E}_{\pi}[G_{t+1} | S_{t+1} = s'] \right] \\ &= \sum_a \pi(a|s) \sum_{s', r} p(s', r | s, a) \left[r + \gamma v_{\pi}(s') \right], \quad \text{for all } s \in \mathcal{S}, \\ &\hspace{15em} \text{TD에서 이용} \end{aligned}$$

Blackjack 게임

Action Space	<code>Discrete(2)</code>
Observation Space	<code>Tuple(Discrete(32), Discrete(11), Discrete(2))</code>
import	<code>gymnasium.make("Blackjack-v1")</code>

Action Space

The action shape is `(1,)` in the range `{0, 1}` indicating whether to stick or hit.

- 0: Stick
- 1: Hit

Rewards

- win game: +1
- lose game: -1
- draw game: 0
- win game with natural blackjack: +1.5 (if `natural` is True) +1 (if `natural` is False)



Monte Carlo Generalized Policy Iteration

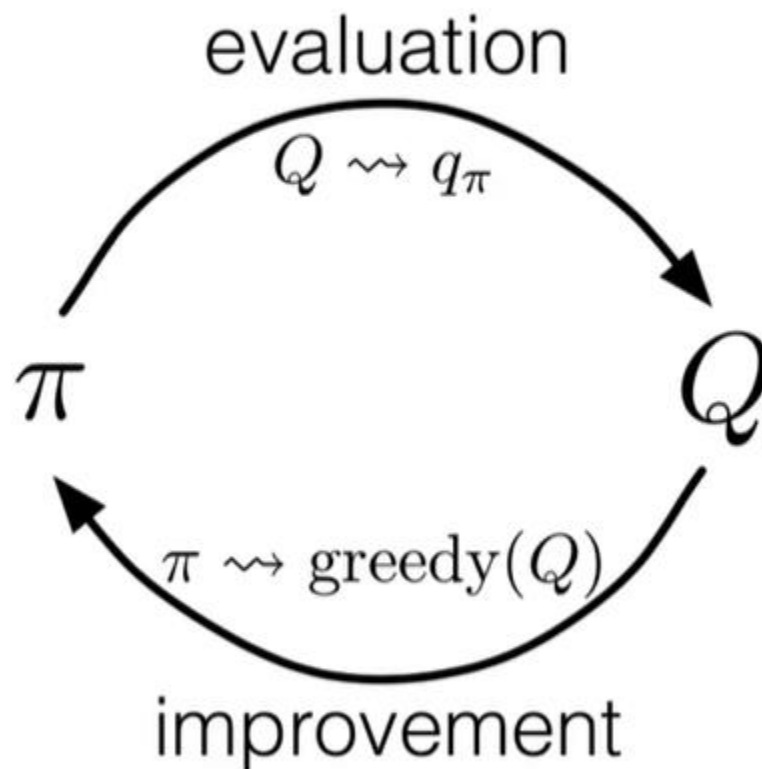
$$\pi_0 \rightarrow \pi_1 \rightarrow \pi_2 \rightarrow \dots$$

Improvement:

$$\pi_{k+1}(s) \doteq \operatorname{argmax}_a q_{\pi_k}(s, a)$$

Evaluation:

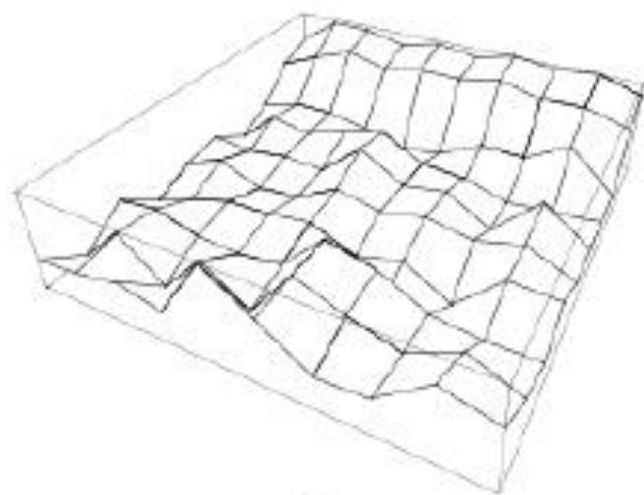
Monte Carlo Prediction



After 10,000 episodes

After 500,000 episodes

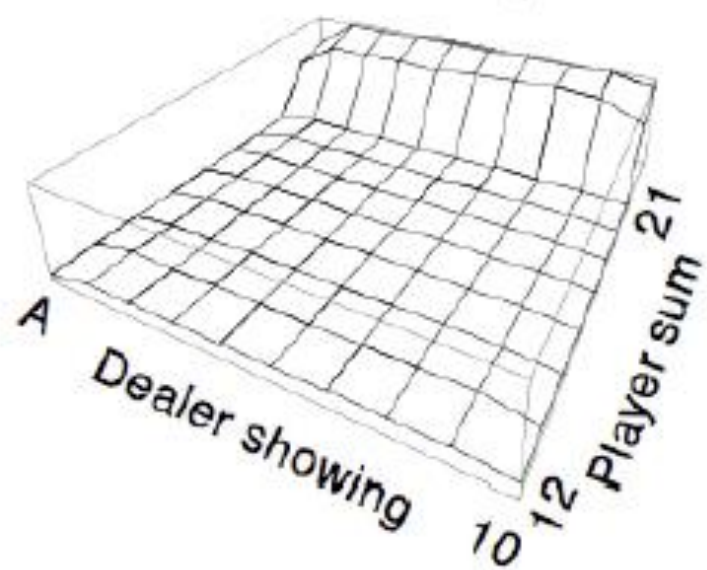
Usable
ace



+1
-1



No
usable
ace



Policy: **stick** if sum of cards ≥ 20 , otherwise **twist**

First-visit MC method

First-visit MC prediction, for estimating $V \approx v_\pi$

Input: a policy π to be evaluated

Initialize:

$V(s) \in \mathbb{R}$, arbitrarily, for all $s \in \mathcal{S}$

$Returns(s) \leftarrow$ an empty list, for all $s \in \mathcal{S}$

Loop forever (for each episode):

Generate an episode following π : $S_0, A_0, R_1, S_1, A_1, R_2, \dots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

Loop for each step of episode, $t = T-1, T-2, \dots, 0$:

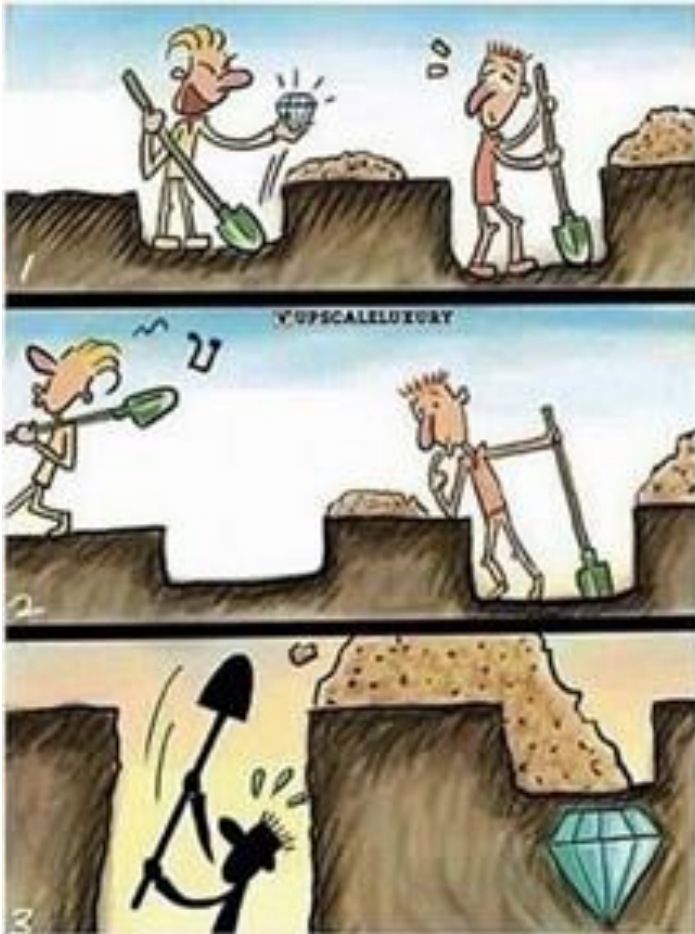
$G \leftarrow G + R_{t+1}$

Unless S_t appears in $S_0, S_1, \dots, S_{t-1}$:

Append G to $Returns(S_t)$

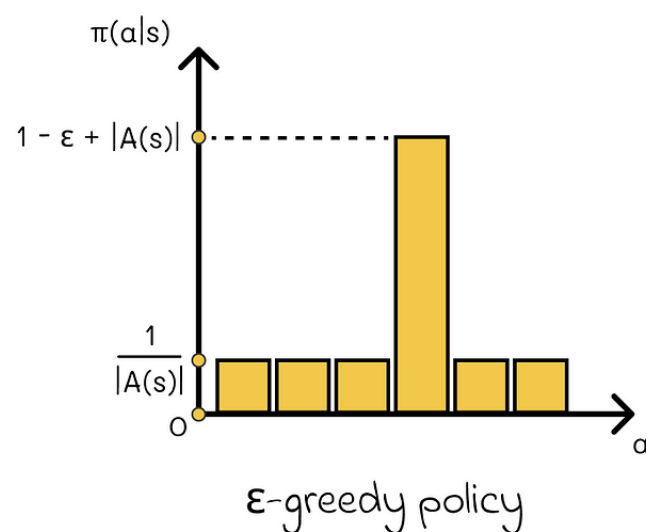
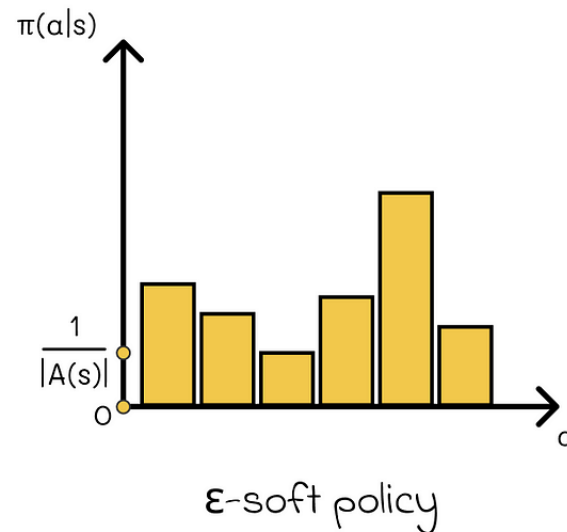
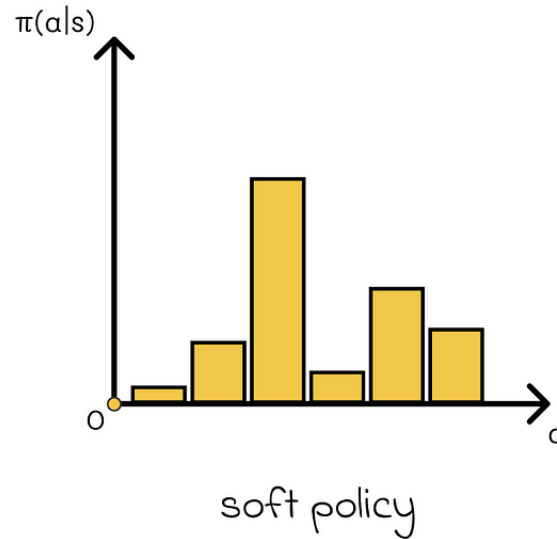
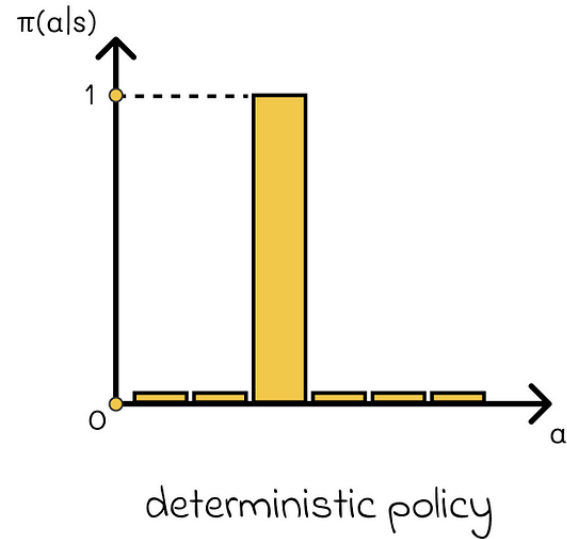
$V(S_t) \leftarrow \text{average}(Returns(S_t))$

Exploration & exploit (decaying ϵ -greedy)



$$A_t \leftarrow \begin{cases} \operatorname{argmax} Q_t(a) & \text{with probability } 1 - \epsilon \\ a \sim \operatorname{Uniform}(\{a_1 \dots a_k\}) & \text{with probability } \epsilon \end{cases}$$

e-soft policy vs e-greedy policy



On-policy first-visit MC control (for ε -soft policies), estimates $\pi \approx \pi_*$

Initialize, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$:

$Q(s, a) \leftarrow$ arbitrary

$Returns(s, a) \leftarrow$ empty list

$\pi(a|s) \leftarrow$ an arbitrary ε -soft policy

Repeat forever:

(a) Generate an episode using π

(b) For each pair s, a appearing in the episode:

$G \leftarrow$ the return that follows the first occurrence of s, a

Append G to $Returns(s, a)$

$Q(s, a) \leftarrow \text{average}(Returns(s, a))$

(c) For each s in the episode:

$A^* \leftarrow \arg \max_a Q(s, a)$

(with ties broken arbitrarily)

For all $a \in \mathcal{A}(s)$:

$$\pi(a|s) \leftarrow \begin{cases} 1 - \varepsilon + \varepsilon/|\mathcal{A}(s)| & \text{if } a = A^* \\ \varepsilon/|\mathcal{A}(s)| & \text{if } a \neq A^* \end{cases}$$

Q-learning

$$Q(s, a) = r(s, a) + \max_a Q(s', a)$$

The diagram illustrates the Q-learning equation with three labels and arrows pointing to the corresponding terms in the formula:

- The label "state" has an arrow pointing to the s in $Q(s, a)$.
- The label "action" has an arrow pointing to the a in $Q(s, a)$.
- The label "reward" has an arrow pointing to the $r(s, a)$ term.

Optimal policy π^* and Max Q

$$\text{Max } Q = \max_{a'} Q(s, a')$$

$$\pi^*(s) = \operatorname{argmax}_a Q(s, a)$$

예)

$Q(\text{state, action})$

$Q(S_1, \text{Left}) : 0$

$Q(S_1, \text{Right}) : 0.5$

$Q(S_1, \text{Up}) : 0.3$

$Q(S_1, \text{Down}) : 0.6$

Dummy Q-learning algorithm

For each s, a initialize table entry $\hat{Q}(s, a) \leftarrow 0$

Observe current state s

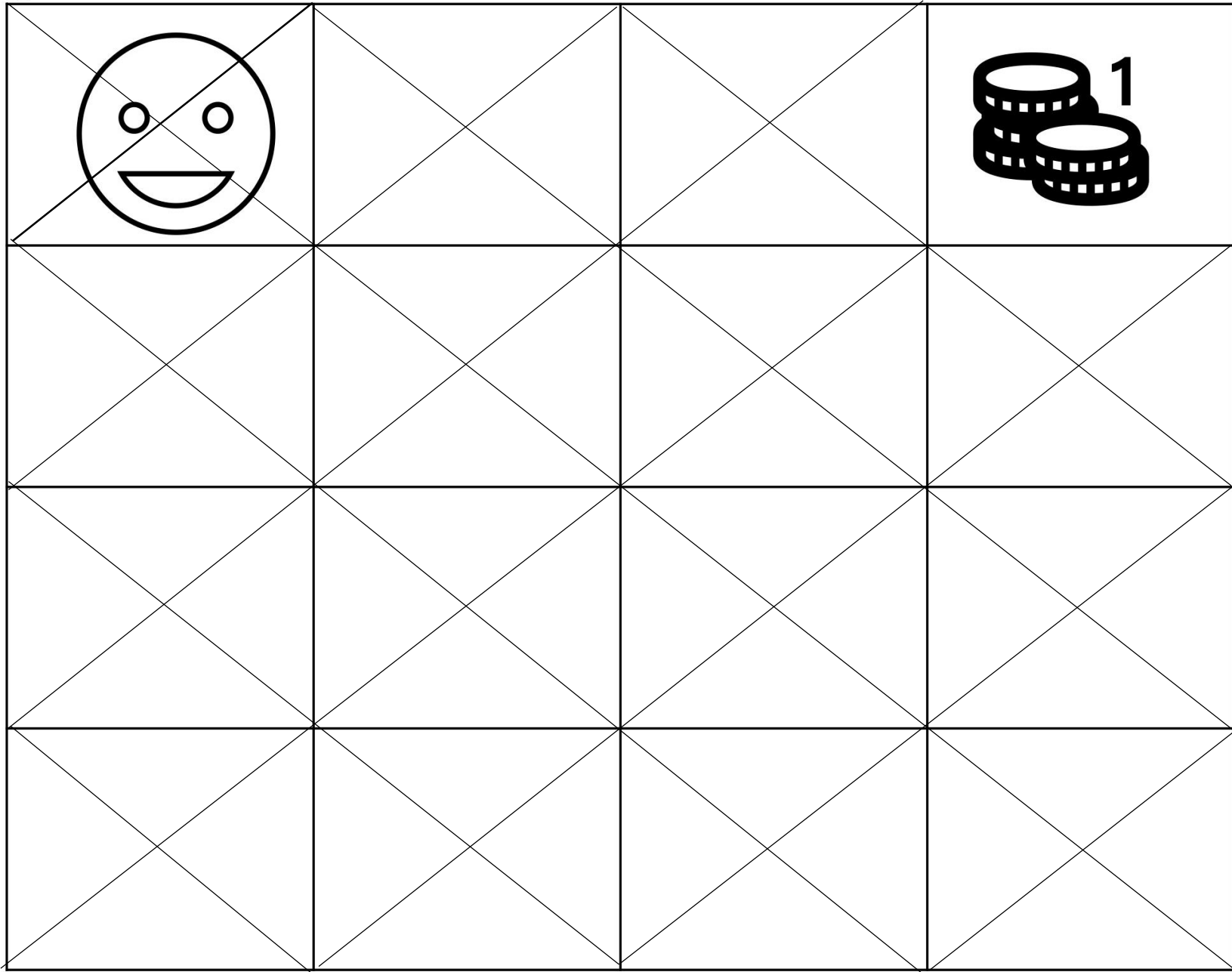
Do forever:

- Select an action a and execute it
- Receive immediate reward r
- Observe the new state s'
- Update the table entry for $\hat{Q}(s, a)$ as follows:

$$\hat{Q}(s, a) \leftarrow r + \max_{a'} \hat{Q}(s', a')$$

- $s \leftarrow s'$

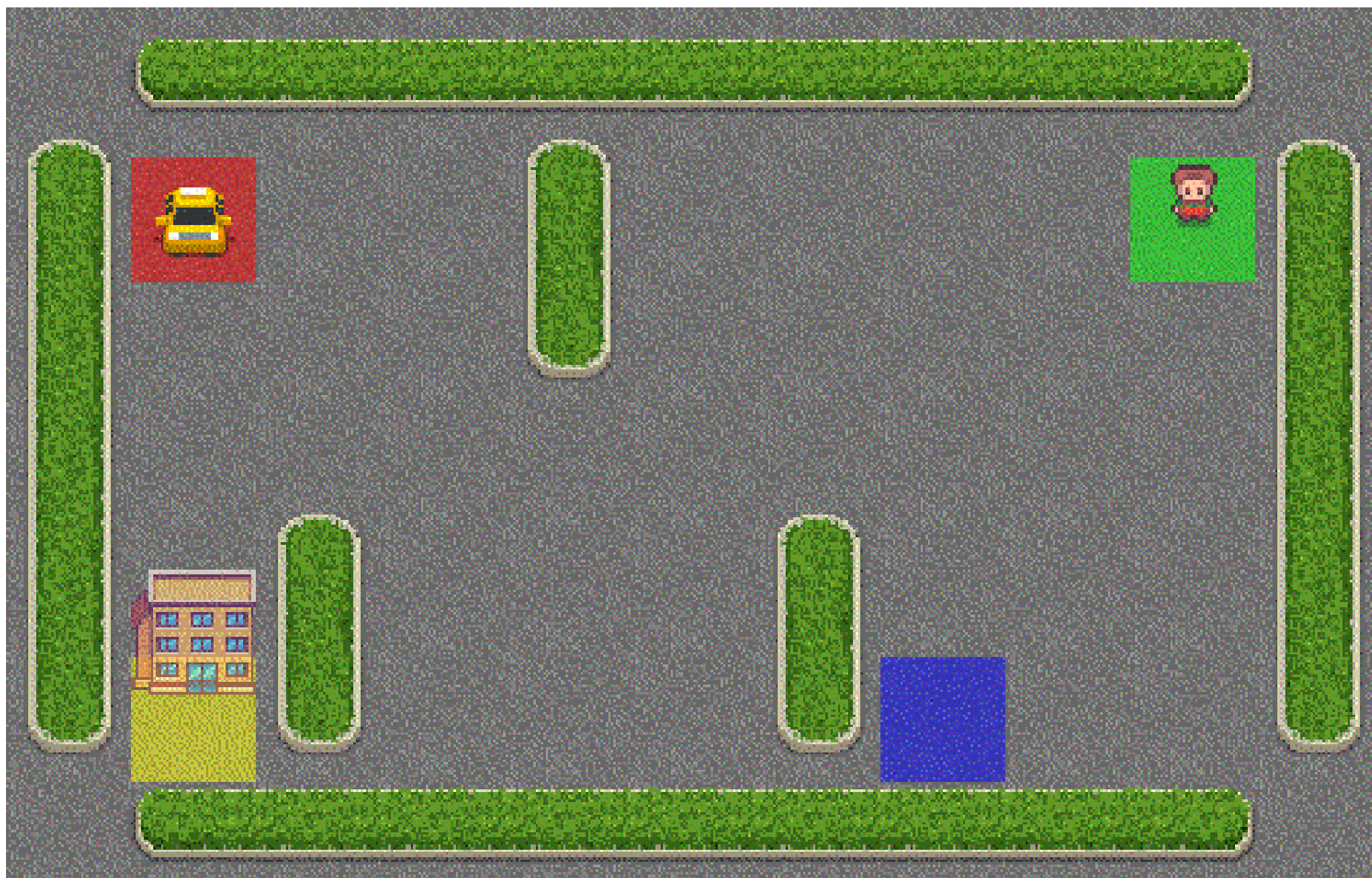
Grid World.



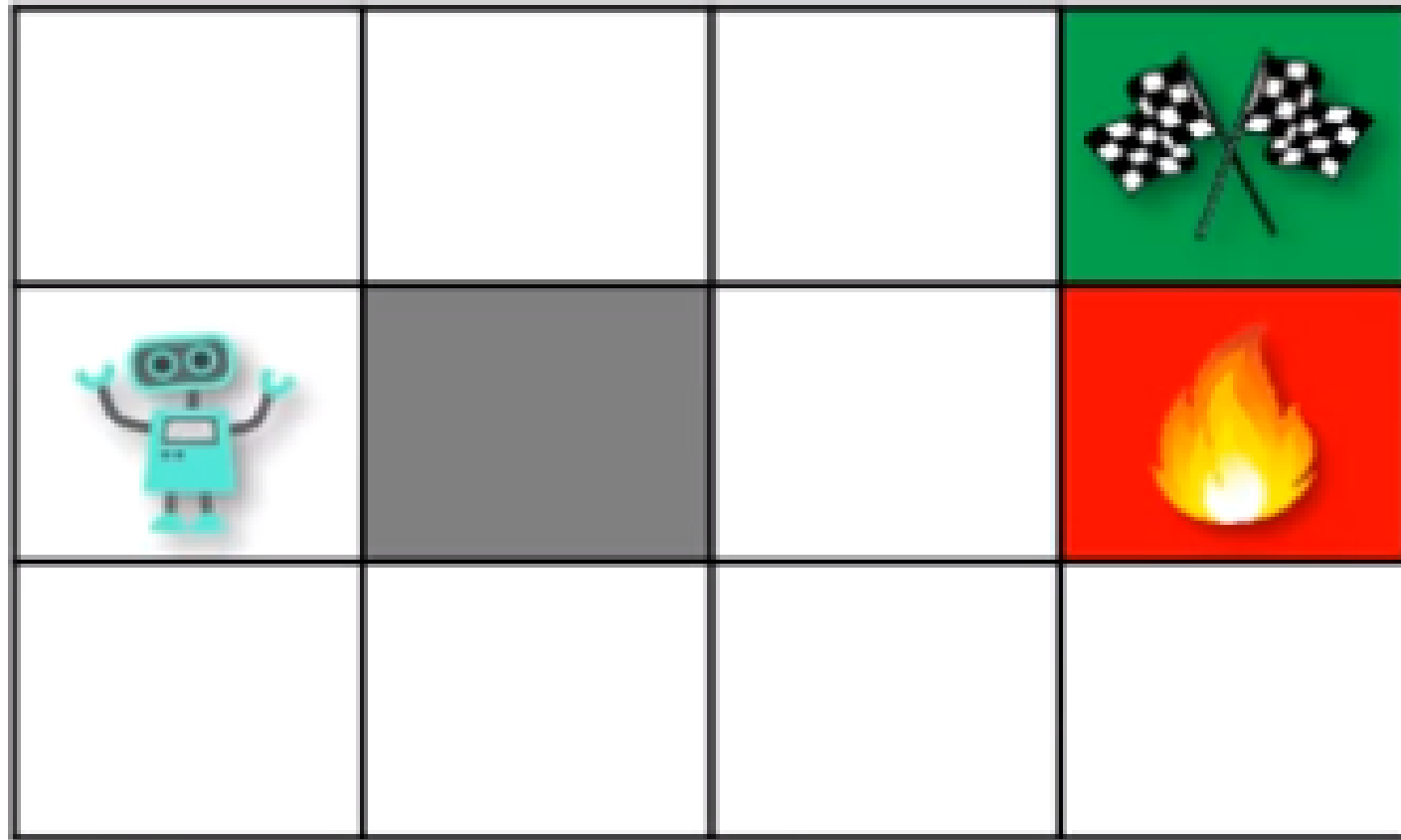
$$Q(s, a) = r(s, a) + \max_a Q(s', a)$$

Diagram illustrating the Q-learning equation with annotations:

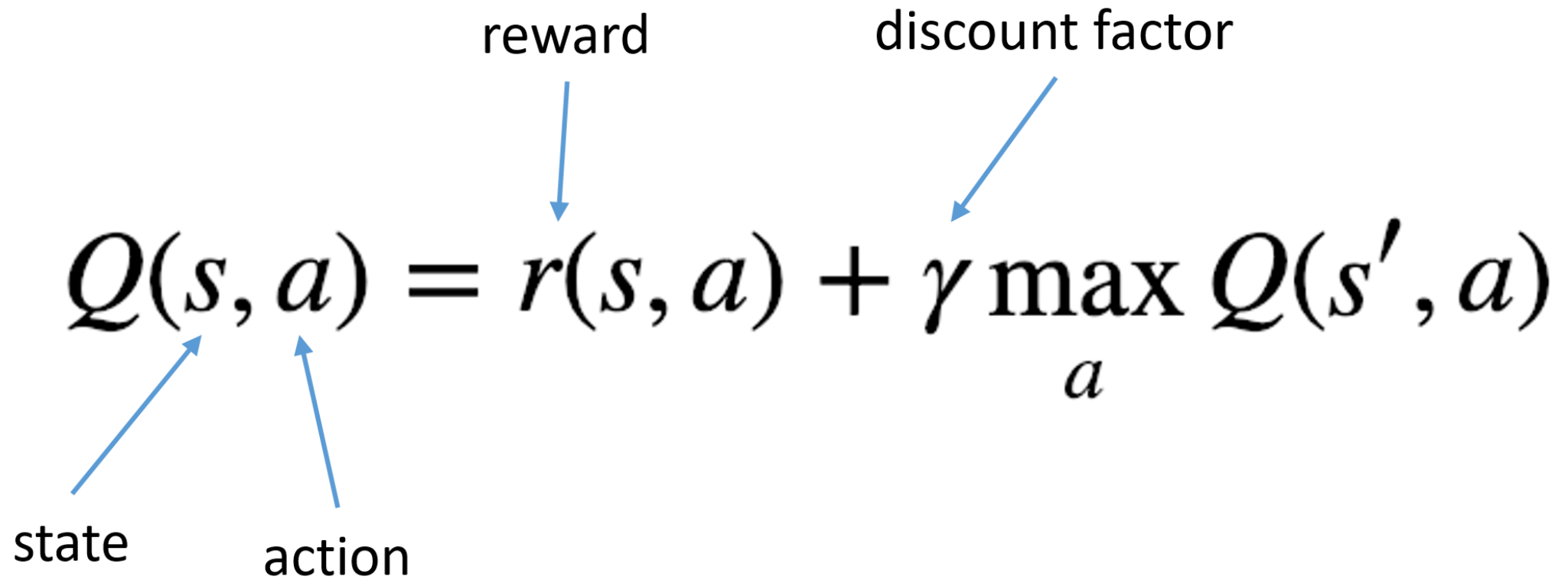
- $Q(s, a)$: state
- a : action
- $r(s, a)$: reward



Q-learning Table with discounting factor γ



Q-learning with discount factor γ



The diagram illustrates the Q-learning equation with labels and arrows pointing to the variables:

$$Q(s, a) = r(s, a) + \gamma \max_a Q(s', a)$$

- reward**: Points to $r(s, a)$
- discount factor**: Points to γ
- state**: Points to s in $Q(s, a)$
- action**: Points to a in $Q(s, a)$

Q-learning in stochastic (real) world

TD learning

α = learning rate

$$Q(s, a) = r(s, a) + \gamma \max_a Q(s', a)$$

Diagram illustrating the Q-learning equation with labels:

- state points to s
- action points to a
- reward points to $r(s, a)$
- discount factor points to γ

$$Q(s, a) \leftarrow (1 - \alpha)Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a')]$$

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

Q-learning algorithm

For each s, a initialize table entry $\hat{Q}(s, a) \leftarrow 0$

Observe current state s

Do forever:

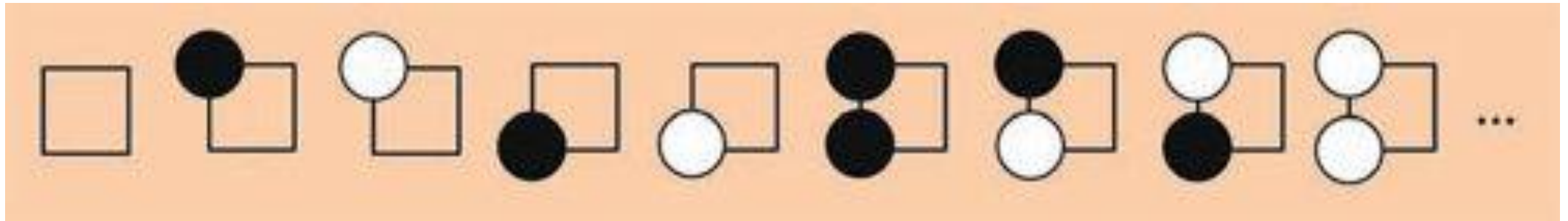
- Select an action a and execute it
- Receive immediate reward r
- Observe the new state s'
- Update the table entry for $\hat{Q}(s, a)$ as follows:

$$Q(s, a) \leftarrow (1 - \alpha)Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a')]$$

- $s \leftarrow s'$

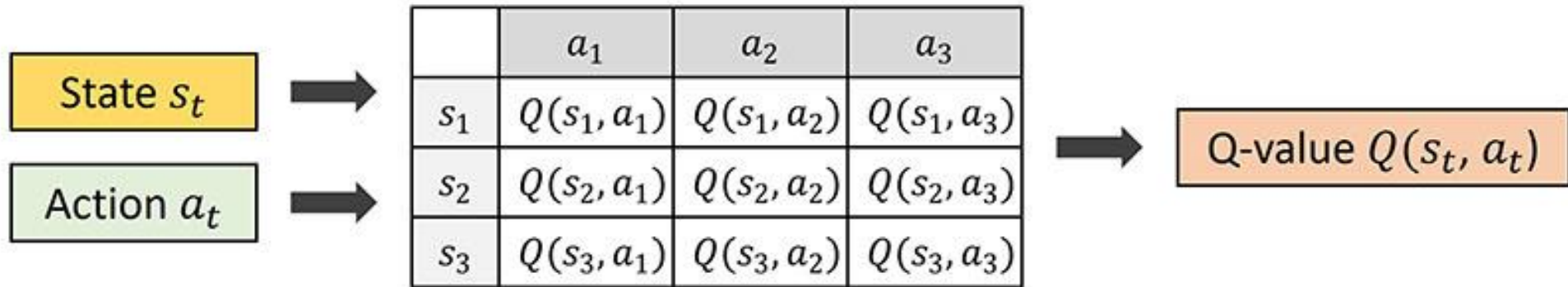
바둑판 state

바둑판의 state의 수 = 10^{170}

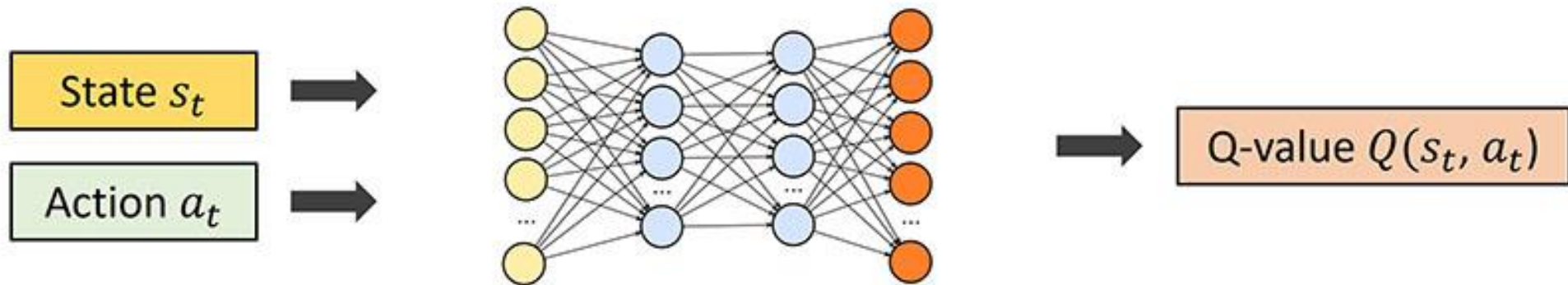


Q - DNN

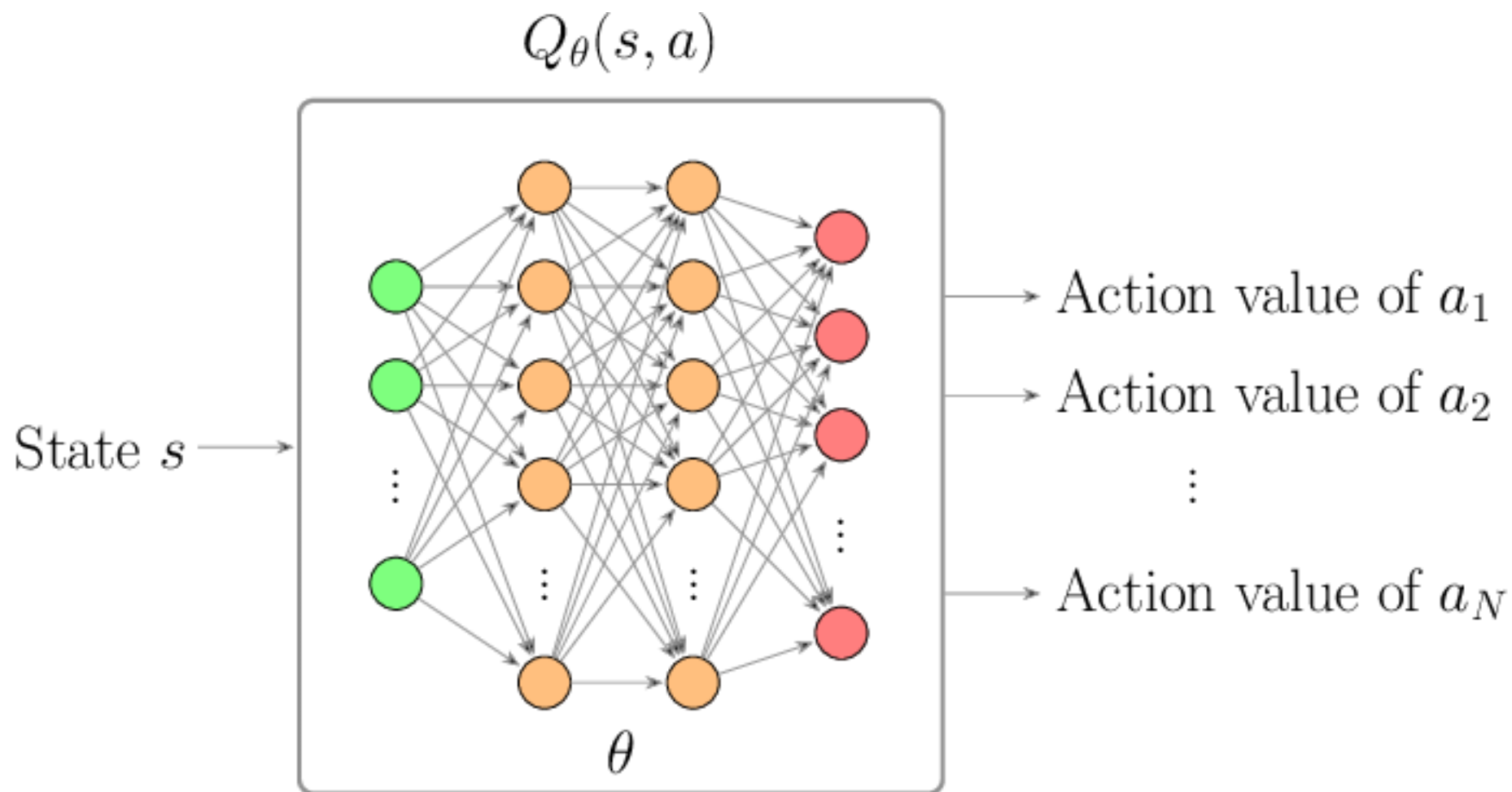
Classic Q-learning



Deep Q-learning



The value-based DQN



Convergence

$\hat{Q}$ denote learner's current approximation to Q .

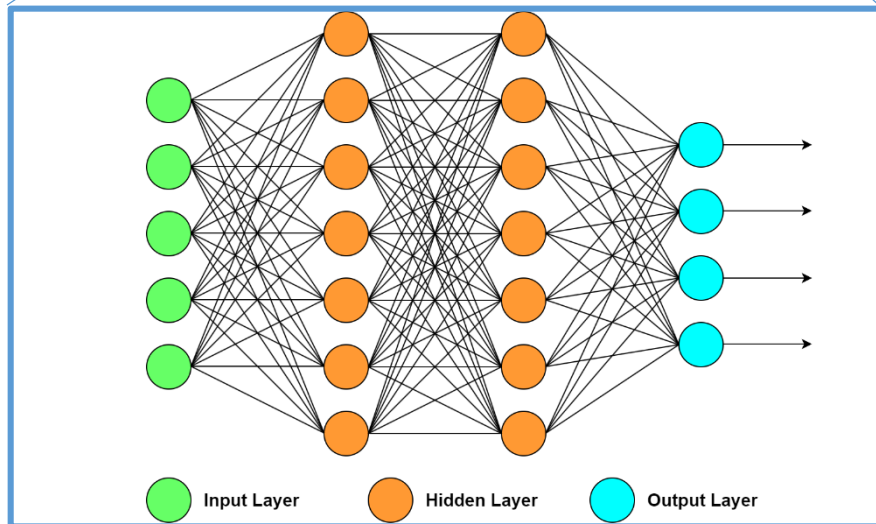
$$\min_{\theta} \sum_{t=0}^T [\hat{Q}(s_t, a_t | \theta) - (r_t + \gamma \max_{a'} \hat{Q}(s_{t+1}, a' | \theta))]^2$$

- ▶ Converges to Q^* using table lookup representation
- ▶ But **diverges** using neural networks due to:
 - ▶ Correlations between samples
 - ▶ Non-stationary targets

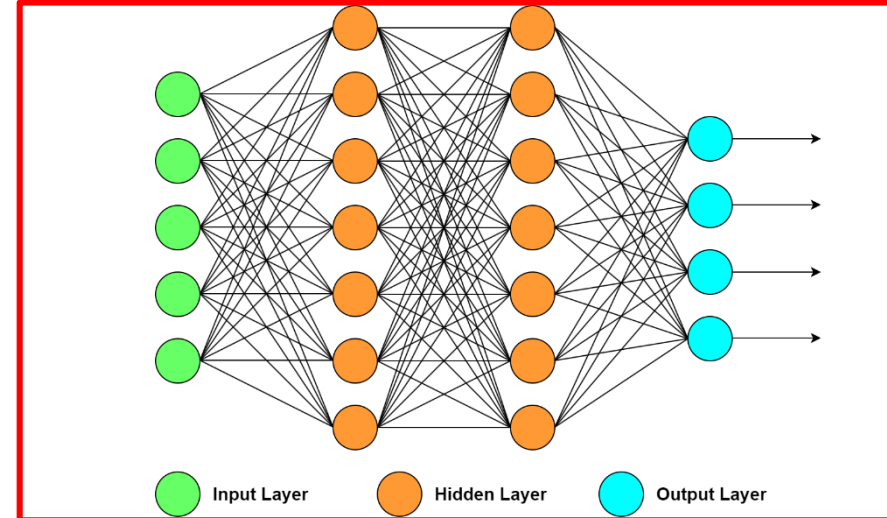
DQN architecture intuition

$$\min_{\theta} \sum_{t=0}^T [\hat{Q}(s_t, a_t | \theta) - (r_t + \gamma \max_{a'} \hat{Q}(s_{t+1}, a' | \theta))]^2$$

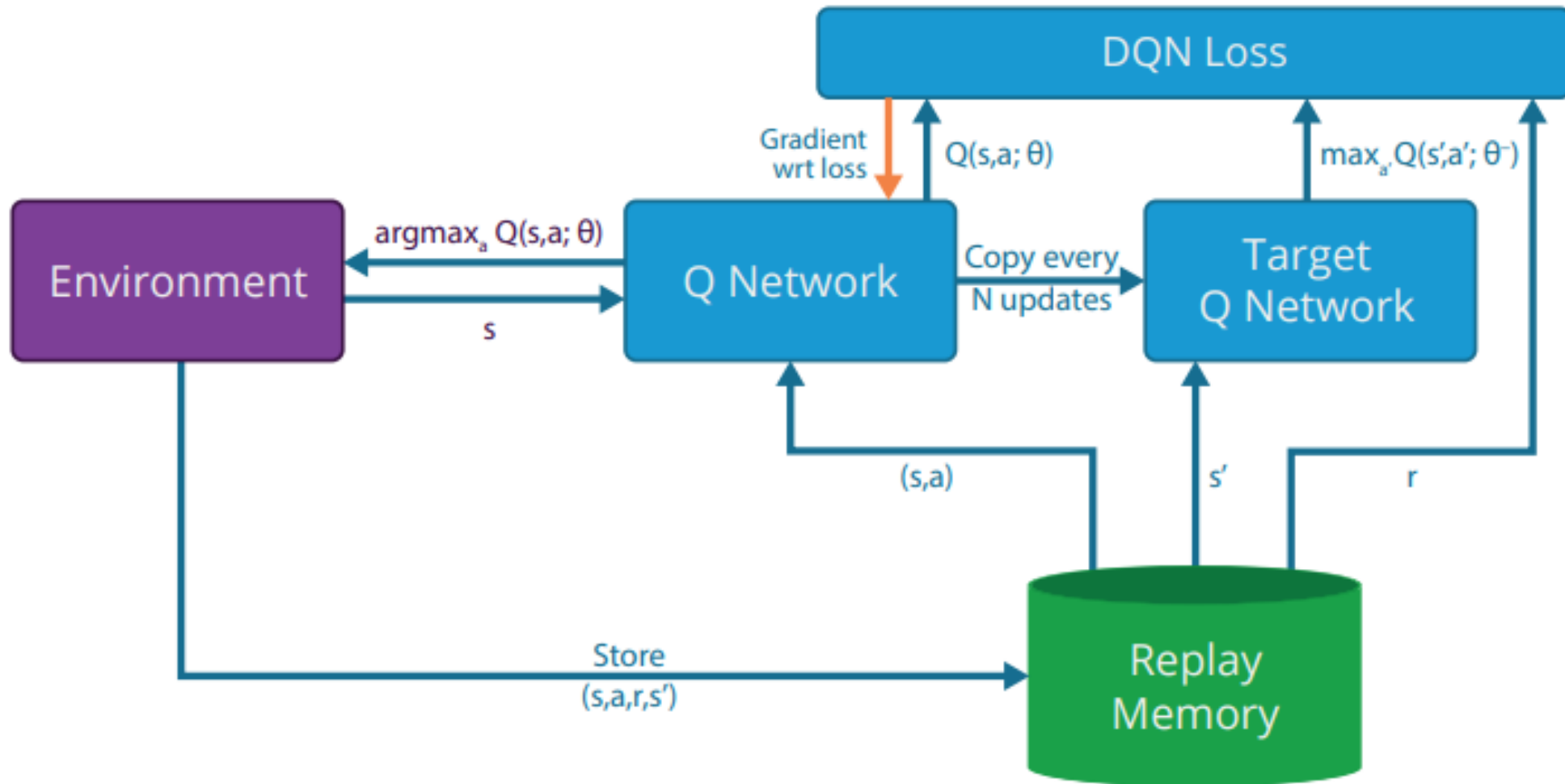
Q-Network



Target-Network



DQN architecture (Deepmind, 2015)



DQN (Deep Q learning, 2015)

Initialize replay memory D to capacity N

Initialize action-value function Q with random weights θ

Initialize target action-value function $\hat{Q}$ with weights $\theta^- = \theta$

Experience Replay

For episode = 1, M do

Initialize sequence $s_1 = \{x_1\}$ and preprocessed sequence $\phi_1 = \phi(s_1)$

For $t = 1, T$ do

With probability ε select a random action a_t

otherwise select $a_t = \operatorname{argmax}_a Q(\phi(s_t), a; \theta)$

Execute action a_t in emulator and observe reward r_t and image x_{t+1}

Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$

Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in D

Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from D

Set $y_j = \begin{cases} r_j & \text{if episode terminates at step } j+1 \\ r_j + \gamma \max_{a'} \hat{Q}(\phi_{j+1}, a'; \theta^-) & \text{otherwise} \end{cases}$

Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ with respect to the network parameters θ

Every C steps reset $\hat{Q} = Q$

Fixed Q-Target

End For

End For